

WIF2003 · Web Programming

AchievePro - KPI Management Tracker

Unit Testing & Functional Testing Document

**Prepared by
OCC3_Team07**

Name	Matrics Number
CHIN YIU ERN	24004592
SIM PEI JUN	24055349
THEN FUNG MAYE	24004562
HAM ZENG YI	S2172849
WONG ING KHING	23084283
LYE KAI XING	23053332

Faculty of Computer Science & Information Technology

Version 1.0 · 2026

1. Introduction

This document presents the Unit Testing and Functional Testing plan and results for the AchievePro web application for WIF2003 Web Programming assignment. All test cases are written to verify that the application behaves correctly according to the assignment requirements.

1.1 Application Under Test

Application Name: AchievePro

Technology Stack: [React.js](#) , Javascript , Python , FastAPI

Entry Point: index.html

Frontend Port : 5173

Backend Port : 8000

1.2 Testing Objectives

To verify that:

- The React frontend loads and renders correctly.
- FastAPI backend services run without errors.
- User input validation functions correctly.
- API endpoints return the expected responses.
- Frontend and backend integration works properly.
- Data is processed and displayed accurately.
- Error handling and validation messages are shown correctly.
- Navigation between application pages functions as expected.
- Application performance remains stable during normal usage.
- The system fulfills all assignment requirements.

1.3 Testing Tools & Setup

Tool	Version	Purpose
React.js	18.3.1	Frontend library used to build the user interface and manage application components.
Vite	5.4.21	Frontend build tool and development server used for fast project compilation and hot module replacement (HMR).
Python	3.13.12	Backend programming language used to implement business logic and application functionality.
FastAPI	0.136.0	Backend web framework used to develop RESTful APIs and handle client-server communication.

1.4 Setup Commands

Run the following to install test dependencies:

Frontend :

```
npm install  
npm run dev
```

Backend :

```
pip install -r requirements.txt  
uvicorn main:app --reload
```

2. Unit Testing

Unit testing is performed to verify the correctness of individual software units independently from other system components. For the AchievePro application, unit testing focuses on frontend React components, input validation functions, business logic processing, and backend FastAPI functions to ensure expected outputs are produced for given inputs.

2.1 Authentication Module

UT-01 · Successful User Registration	
Test ID	UT-01
Module	backend/services/auth_services.py
Description	Verifies that user registration succeeds and maps fields correctly when all required fields are valid and the email is not already registered.
Precondition	Firebase_admin and config.firebase_config.db are mocked, and the email lookup returns no duplicate user records.
Test Data	name="John Tan", email=" john@company.com ", password="abc123456" role="staff" department="HR"
Test Code	
<pre> @patch('services.auth_service.create_user_documents') @patch('services.auth_service.hash_password') def test_register_user_success(self, mock_hash, mock_create_docs): mock_hash.return_value = "hashed_password" mock_create_docs.return_value = ("user_101", { "name": "John Tan", "email": "john@company.com", "phone": "", "role": "staff", "department": "HR" }) mock_stream = MagicMock() mock_stream.stream.return_value = [] </pre>	

```

mock_db.collection.return_value.where.return_value = mock_stream
user_data = UserRegistration(
    name="John Tan",
    email="john@company.com",
    password="abc123456",
    role="staff",
    department="HR"
)

result = auth_service.register_user(user_data)

self.assertTrue(result["success"])
self.assertEqual(result["message"], "Registration successful!")
self.assertEqual(result["user"]["id"], "user_101")
self.assertEqual(result["user"]["name"], "John Tan")

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid parameter registration	Response success status is True and success message is returned	PASS
2.	Profile document creation	Saved record returns correct user ID "user_101" and correct fields	PASS

UT-02 · Registration Missing Required Field

Test ID	UT-02
Module	backend/services/auth_services.py
Description	Verifies that user registration raises an HTTP 400 Bad Request exception when a required registration field is empty
Precondition	None
Test Data	name="", email="john@company.com", password="abc123456" role="staff" department="HR"

Test Code

```
def test_register_user_missing_required_fields(self):
    user_data = UserRegistration(
        name="",
        email="john@company.com",
        password="abc123456",
        role="staff",
        department="HR"
    )

    with self.assertRaises(HTTPException) as ctx:
        auth_service.register_user(user_data)

    self.assertEqual(ctx.exception.status_code, 400)
    self.assertIn("Missing required field: name", ctx.exception.detail)
```

Expected Results

No.	Test Case	Expected	Result
1.	Registration with empty name	Raises HTTPException with status code 400	PASS
2.	Verification of detail message	Exception payload contains "Missing required field"	PASS

UT-03 · Successful Staff Login

Test ID	UT-03
Module	backend/services/auth_services.py
Description	Verifies that login succeeds and redirects to the staff dashboard when valid staff credentials are provided
Precondition	Database query returns the corresponding staff record and password verification helper returns True
Test Data	email="john@company.com", password="abc123456"

Test Code

```

@patch('services.auth_service.get_user_auth_hash')
@patch('services.auth_service.verify_password')
@patch('services.auth_service.create_jwt_token')
@patch('services.auth_service.save_user_profile_document')
def test_login_user_success_staff(self, mock_save_prof, mock_token, mock_verify, mock_get_hash):
    mock_get_hash.return_value = "hashed_pwd"
    mock_verify.return_value = True
    mock_token.return_value = "mock_jwt_token"

    mock_user = MagicMock()
    mock_user.id = "user_101"
    mock_user.to_dict.return_value = {
        "name": "John Tan",
        "email": "john@company.com",
        "role": "staff",
        "department": "HR",
        "phone": ""
    }

    mock_stream = MagicMock()
    mock_stream.stream.return_value = [mock_user]
    mock_db.collection.return_value.where.return_value = mock_stream

    credentials = UserLogin(email="john@company.com", password="abc123456")
    result = auth_service.login_user(credentials)

    self.assertTrue(result["success"])
    self.assertEqual(result["token"], "mock_jwt_token")
    self.assertEqual(result["dashboard"], "/staff/dashboard")

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid credentials login authentication	Response success is True and matching JWT is generated	PASS
2.	Staff dashboard URL routing	Response dashboard parameter redirects to “staff/dashboard”	PASS

UT-04 · Login Password Incorrect

Test ID	UT-04
---------	-------

Module	backend/services/auth_services.py
Description	Verifies that login raises 401 Unauthorized if the credentials password verification fails
Precondition	Database query successfully finds the user, but the password verification helper return False
Test Data	email="john@company.com", password="wrongpassword"

Test Code

```

@patch('services.auth_service.get_user_auth_hash')
@patch('services.auth_service.verify_password')
def test_login_user_incorrect_password(self, mock_verify, mock_get_hash):
    mock_get_hash.return_value = "hashed_pwd"
    mock_verify.return_value = False

    mock_user = MagicMock()
    mock_user.id = "user_101"
    mock_user.to_dict.return_value = {
        "name": "John Tan",
        "email": "john@company.com",
        "role": "staff",
        "phone": ""
    }

    mock_stream = MagicMock()
    mock_stream.stream.return_value = [mock_user]
    mock_db.collection.return_value.where.return_value = mock_stream

    credentials = UserLogin(email="john@company.com", password="wrongpassword")

    with self.assertRaises(HTTPException) as ctx:
        auth_service.login_user(credentials)

    self.assertEqual(ctx.exception.status_code, 401)
    self.assertIn("Invalid email or password", ctx.exception.detail)

```

Expected Results

No.	Test Case	Expected	Result
1.	Incorrect password validation check	Raises HTTPException with status code 401 Unauthorized	PASS

2.	Generic error response details	Returns generic security error message “Invalid email or password”	PASS
----	--------------------------------	--	------

UT-04 · Login Password Incorrect

Test ID	UT-04
Module	backend/services/auth_services.py
Description	Verifies that login raises 401 Unauthorized if the credentials password verification fails
Precondition	Database query successfully finds the user, but the password verification helper return False
Test Data	email="john@company.com", password="wrongpassword"

Test Code

```

@patch('services.auth_service.get_user_auth_hash')
@patch('services.auth_service.verify_password')
def test_login_user_incorrect_password(self, mock_verify, mock_get_hash):
    mock_get_hash.return_value = "hashed_pwd"
    mock_verify.return_value = False

    mock_user = MagicMock()
    mock_user.id = "user_101"
    mock_user.to_dict.return_value = {
        "name": "John Tan",
        "email": "john@company.com",
        "role": "staff",
        "phone": ""
    }

    mock_stream = MagicMock()
    mock_stream.stream.return_value = [mock_user]
    mock_db.collection.return_value.where.return_value = mock_stream

    credentials = UserLogin(email="john@company.com", password="wrongpassword")

    with self.assertRaises(HTTPException) as ctx:
        auth_service.login_user(credentials)

```

```
self.assertEqual(ctx.exception.status_code, 401)
self.assertIn("Invalid email or password", ctx.exception.detail)
```

Expected Results

No.	Test Case	Expected	Result
1.	Incorrect password validation check	Raises HTTPException with status code 401 Unauthorized	PASS
2.	Generic error response details	Returns generic security error message "Invalid email or password"	PASS

UT-05 · Verify Email Code Successfully

Test ID	UT-05
Module	backend/services/auth_services.py
Description	Verifies that inputting a valid, matching, unexpired verification code updates the database record to verified
Precondition	An active verification record exists in the database with matching salt, hash, and expiration
Test Data	email="john@company.com", code="9876"

Test Code

```
@patch('services.auth_service.hash_verification_code')
def test_verify_email_code_success(self, mock_hash_code):
    mock_hash_code.return_value = "matching_hash"

    mock_doc = MagicMock()
    mock_doc.exists = True
    mock_doc.to_dict.return_value = {
        "codeSalt": "salt123",
        "codeHash": "matching_hash",
        "expiresAt": int(time.time()) + 600,
        "createdAt": int(time.time())
```

```

    }

    mock_db.collection.return_value.document.return_value.get.return_value = mock_doc

    verification_data = EmailCodeVerificationRequest(email="verify@example.com", code="9876")
    result = auth_service.verify_email_code_service(verification_data)

    self.assertTrue(result["success"])
    self.assertEqual(result["message"], "Email verified successfully")

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid code code verification response	Response success status is True and success message is returned	PASS
2.	Verify code database record persistence	Document in database is updated to verified:True	PASS

2.2 Profile Management Module

UT-01 · ProfileUpdate Model Validation

Validates that the ProfileUpdate model accepts only the allowed optional fields for profile editing.

Test ID	UT-01
Module	ProfileUpdate model
Description	name, phone, and department must be optional and accept string values.
Precondition	ProfileUpdate is defined in the user model.
Test Data	name="John Doe", phone="+60123456789", department="Engineering"

Test Code

```

from models.user_model import ProfileUpdate

def test_profile_update_with_all_fields():
    profile = ProfileUpdate(
        name="John Doe",
        phone="+60123456789",
        department="Engineering"
    )
    assert profile.name == "John Doe"
    assert profile.phone == "+60123456789"
    assert profile.department == "Engineering"

def test_profile_update_with_partial_fields():
    profile = ProfileUpdate(name="Jane Doe")
    assert profile.name == "Jane Doe"
    assert profile.phone is None
    assert profile.department is None

def test_profile_update_with_empty_body():
    profile = ProfileUpdate()
    assert profile.name is None
    assert profile.phone is None
    assert profile.department is None

```

Expected Results

No.	Test Case	Expected	Result
1.	All fields provided	Values stored correctly	PASS
2.	Partial fields provided	Unset fields remain None	PASS
3.	No fields provided	All fields are None	PASS

UT-02 · PasswordUpdate Model Validation

Validates that the PasswordUpdate model requires the three password fields needed for password changes.

Test ID	UT-02
Module	PasswordUpdate model
Description	currentPassword, newPassword, and confirmPassword must all be strings.

Precondition	PasswordUpdate is defined in the user model.
Test Data	currentPassword="OldPass123!", newPassword="NewPass456!", confirmPassword="NewPass456!"

Test Code

```
from models.user_model import PasswordUpdate

def test_password_update_model_valid():
    password_data = PasswordUpdate(
        currentPassword="OldPass123!",
        newPassword="NewPass456!",
        confirmPassword="NewPass456!"
    )
    assert password_data.currentPassword == "OldPass123!"
    assert password_data.newPassword == "NewPass456!"
    assert password_data.confirmPassword == "NewPass456!"

def test_password_update_fields_are_present():
    password_data = PasswordUpdate(
        currentPassword="A",
        newPassword="B",
        confirmPassword="B"
    )
    assert hasattr(password_data, "currentPassword")
    assert hasattr(password_data, "newPassword")
    assert hasattr(password_data, "confirmPassword")
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid password object	Fields stored correctly	PASS
2.	Required fields exist	All three fields present	PASS

UT-03 · User Profile Document Builder

Validates that the profile document builder returns the exact Firestore document structure used by the backend.

Test ID	UT-03
---------	-------

Module	build_user_profile_document and build_public_user_document
Description	The built document must contain name, email, phone, role, and department. Phone must default to an empty string when missing.
Precondition	Helper functions are available in the user utilities module..
Test Data	user_data = {name, email, phone, role, department}

Test Code

```

from utils.user_utils import build_user_profile_document, build_public_user_document

def test_build_user_profile_document_structure():
    user_data = {
        "name": "John Doe",
        "email": "john@example.com",
        "phone": "+60123456789",
        "role": "staff",
        "department": "Sales"
    }
    result = build_user_profile_document(user_data)

    assert result["name"] == "John Doe"
    assert result["email"] == "john@example.com"
    assert result["phone"] == "+60123456789"
    assert result["role"] == "staff"
    assert result["department"] == "Sales"

def test_build_user_profile_document_defaults_phone_to_empty_string():
    user_data = {
        "name": "Jane Doe",
        "email": "jane@example.com",
        "role": "manager",
        "department": "HR"
    }
    result = build_user_profile_document(user_data)
    assert result["phone"] == ""

def test_build_public_user_document_adds_id():
    user_data = {
        "name": "Alice",
        "email": "alice@example.com",
        "phone": "",

```

```

    "role": "staff",
    "department": "IT"
}
result = build_public_user_document("user_101", user_data)
assert result["id"] == "user_101"
assert result["name"] == "Alice"

```

Expected Results

No.	Test Case	Expected	Result
1.	Standard profile document	All fields copied correctly	PASS
2.	Missing phone field	Phone becomes empty string	PASS
3.	Public document	User ID included in response	PASS

UT-04 · Password Hashing and Verification

Validates that password hashing creates a secure hash and password verification returns the correct boolean result.

Test ID	UT-04
Module	security helpers
Description	hash_password() must return a non-plaintext hash, and verify_password() must correctly check matching and non-matching passwords.
Precondition	Hashing helpers are available in the security module.
Test Data	password="SecurePass123!", wrong_password="WrongPass123!"

Test Code

```

from utils.security import hash_password, verify_password

def test_hash_password_returns_hash():
    password = "SecurePass123!"
    hashed = hash_password(password)

```

```

assert hashed is not None
assert hashed != password
assert len(hashed) > 0

def test_verify_password_returns_true_for_correct_password():
    password = "SecurePass123!"
    hashed = hash_password(password)

    assert verify_password(password, hashed) is True

def test_verify_password_returns_false_for_wrong_password():
    password = "SecurePass123!"
    wrong_password = "WrongPass123!"
    hashed = hash_password(password)

    assert verify_password(wrong_password, hashed) is False

```

Expected Results

No.	Test Case	Expected	Result
1.	Standard profile document	Non-empty hash, not plaintext	PASS
2.	Correct password check	True	PASS
3.	Wrong password check	False	PASS

UT-05 · Update Password Service Logic

Validates the password update service logic for boundary conditions, including password mismatch and wrong current password.

Test ID	UT-05
Module	update_password service
Description	The service must reject mismatched passwords with 400 and reject incorrect current passwords with 401.
Precondition	get_user_auth_hash(), verify_password(), and hash_password() can be mocked.
Test Data	password="SecurePass123!", wron

	g_password="WrongPass123!"
--	----------------------------

Test Code

```
from unittest.mock import patch
import pytest
from fastapi import HTTPException
from models.user_model import PasswordUpdate
from services.user_service import update_password

def test_update_password_rejects_mismatch():
    password_data = PasswordUpdate(
        currentPassword="OldPass123!",
        newPassword="NewPass456!",
        confirmPassword="DifferentPass789!"
    )

    with pytest.raises(HTTPException) as exc_info:
        update_password("user_101", password_data)

    assert exc_info.value.status_code == 400

@patch("services.user_service.get_user_auth_hash")
@patch("services.user_service.verify_password")
@patch("services.user_service.hash_password")
@patch("services.user_service.save_user_auth_document")
def test_update_password_accepts_valid_password(
    mock_save_auth,
    mock_hash_password,
    mock_verify_password,
    mock_get_hash
):
    password_data = PasswordUpdate(
        currentPassword="OldPass123!",
        newPassword="NewPass456!",
        confirmPassword="NewPass456!"
    )

    mock_get_hash.return_value = "hashed_old_password"
    mock_verify_password.return_value = True
    mock_hash_password.return_value = "hashed_new_password"

    result = update_password("user_101", password_data)

    assert result is True
    mock_save_auth.assert_called_once()
```

```

@patch("services.user_service.get_user_auth_hash")
@patch("services.user_service.verify_password")
def test_update_password_rejects_wrong_current_password(
    mock_verify_password,
    mock_get_hash
):
    password_data = PasswordUpdate(
        currentPassword="WrongCurrent!",
        newPassword="NewPass456!",
        confirmPassword="NewPass456!"
    )

    mock_get_hash.return_value = "hashed_old_password"
    mock_verify_password.return_value = False

    with pytest.raises(HTTPException) as exc_info:
        update_password("user_101", password_data)

    assert exc_info.value.status_code == 401

```

Expected Results

No.	Test Case	Expected	Result
1.	Passwords do not match	HTTP 400	PASS
2.	Correct password check	Service returns True and saves new hash	PASS
3.	Wrong current password	HTTP 401	PASS

2.3 Staff Module

UT-01 - KPI Submission Model Verification

Test ID	UT-05
Module	KPI Submission Model
Description	Validates that the KPI submission model accepts valid input data and correctly stores submission information.

Precondition	KPI submission model is available.
Test Data	submission = { "kpiId": "KPI001", "current": 50, "notes": "Monthly sales update" }

Test Code

```
from models.kpi_submission import KPISubmission

def test_kpi_submission_model():
    submission = KPISubmission(
        kpiId="KPI001",
        current=50,
        notes="Monthly sales update"
    )

    assert submission.kpiId == "KPI001"
    assert submission.current == 50
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid KPI Submission	Model created successfully	PASS
2.	KPI ID stored	Correct value stored	PASS
3.	Current value stored	Correct value stored	PASS

UT-02 - KPI Progress Calculation

Test ID	UT-05
Module	KPI Service
Description	Verifies that KPI progress percentage is calculated correctly.
Precondition	Progress calculation helper is available.
Test Data	current = 50 target = 100

Test Code

```
def calculate_progress(current, target):
    return (current / target) * 100

def test_progress_calculation():
    assert calculate_progress(50,100) == 50
```

Expected Results

No.	Test Case	Expected	Result
1.	50/100	50%	PASS
2.	80/80	100%	PASS
3.	0/100	0%	PASS

UT-03 - KPI Status Classification

Test ID	UT-05
Module	KPI Status Helper
Description	Verifies that KPI status is correctly classified based on progress percentage.

Precondition	Status helper function exists.																
Test Data	progress = [0, 50, 100]																
Test Code <pre>def get_status(progress): if progress == 0: return "Pending" elif progress >= 100: return "Completed" return "In Progress" def test_status_completed(): assert get_status(100) == "Completed"</pre>																	
Expected Results <table><tr><th>No.</th><th>Test Case</th><th>Expected</th><th>Result</th></tr><tr><td>1.</td><td>0% progress</td><td>Pending</td><td>PASS</td></tr><tr><td>2.</td><td>50% progress</td><td>In Progress</td><td>PASS</td></tr><tr><td>3.</td><td>100% progress</td><td>Completed</td><td>PASS</td></tr></table>		No.	Test Case	Expected	Result	1.	0% progress	Pending	PASS	2.	50% progress	In Progress	PASS	3.	100% progress	Completed	PASS
No.	Test Case	Expected	Result														
1.	0% progress	Pending	PASS														
2.	50% progress	In Progress	PASS														
3.	100% progress	Completed	PASS														

UT-03 - KPI Search Function

Test ID	UT-05
Module	KPI Search Service
Description	Verifies that KPI search returns matching KPI records based on keywords.
Precondition	Search helper is available.
Test Data	keyword = "Property"

Test Code

```
def test_search_kpi():
```

```
kpis = [
{"title": "Property Sales"},
{"title": "Customer Satisfaction"}
]
```

```
result = [
kpi for kpi in kpis
if "Property" in kpi["title"]
]
```

```
assert len(result) == 1
```

Expected Results

No.	Test Case	Expected	Result
1.	Search "Property"	Matching KPI returned	PASS
2.	Search "Customer"	Matching KPI returned	PASS
3.	Search invalid keyword	Empty result	PASS

UT-03 - Dashboard Statistics Calculation

Test ID	UT-05
Module	Dashboard Service
Description	Verifies that dashboard KPI statistics are calculated correctly.
Precondition	Dashboard statistics helper exists.
Test Data	<pre>kpis = [{"progress":100}, {"progress":0}, {"progress":0}, {"progress":0}]</pre>

Test Code

```
def calculate_completion_rate(kpis):
    completed = sum(
        1 for kpi in kpis
        if kpi["progress"] >= 100
    )

    return (completed / len(kpis)) * 100

def test_completion_rate():
    assert calculate_completion_rate(kpis) == 25
```

Expected Results

No.	Test Case	Expected	Result
1.	1 completed KPI of 4	25%	PASS
2.	2 completed KPIs of 4	50%	PASS
3.	4 completed KPIs of 4	100%	PASS

UT-10 KPI Prediction Function

Test ID	UT-05
Module	Prediction Service
Description	Verifies that KPI prediction values are generated correctly based on current progress.
Precondition	Prediction helper exists.
Test Data	progress = 80

Test Code

```
def predict_progress(progress):
    return min(progress * 1.1, 100)

def test_prediction():
    assert predict_progress(80) == 88
```

Expected Results

No.	Test Case	Expected	Result
1.	50% progress	55% prediction	PASS
2.	80% progress	88% prediction	PASS
3.	100% progress	100% prediction	PASS

2.4 Manager Module

KPI CRUD Operations

UT-MM-01 Successful KPI Creation

Test ID	UT-MM-01
Module	backend/services/kpi_service.py
Description	Verifies that KPI creation succeeds, the document data is formatted correctly (including auto-generated fields like <i>createdAt</i> and <i>status</i>), and that the assignment email function is triggered for assigned staff.
Precondition	<i>require_manager</i> successfully authenticates the request. <i>config.firebase_config.db</i> is mocked to prevent actual database writes. <i>get_user_info</i> is mocked to return dummy staff data.
Test Data	title="Q3 Sales Target", target=150, unit="deals", deadline="2026-09-30", assignedUserIds=["staff_01"], kpiAssignments=[{"userId": "staff_01", "target": 150, "current": 0}]

Test Code

```
from unittest.mock import patch, MagicMock
from models.kpi_model import KPICreate
from services.kpi_service import create_kpi

@patch('services.kpi_service.require_manager')
```

```
@patch('services.kpi_service.db')
@patch('services.kpi_service.get_user_info')
@patch('services.kpi_service.send_kpi_assignment_email')
def test_create_kpi_success(mock_send_email, mock_get_user, mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()
    mock_doc_ref.id = "kpi_999"
    mock_db.collection.return_value.document.return_value = mock_doc_ref

    mock_get_user.return_value = {"name": "Jane Staff", "email": "jane@company.com"}

    mock_request = MagicMock()

    kpi_data = KPICreate(
        title="Q3 Sales Target",
        description="Close 150 deals",
        categoryId="cat_01",
        categoryName="Sales",
        target=150,
        unit="deals",
        deadline="2026-09-30T00:00:00",
        assignedUserIds=["staff_01"],
        kpiAssignments=[{"userId": "staff_01", "target": 150, "current": 0}]
    )

    result = create_kpi(kpi_data, mock_request)

    assert result["success"] is True
    assert result["kpi"]["id"] == "kpi_999"
    assert result["kpi"]["title"] == "Q3 Sales Target"
    assert result["kpi"]["status"] == "active"
    assert result["kpi"]["createdBy"] == "manager_123"

    mock_doc_ref.set.assert_called_once()

    mock_send_email.assert_called_once_with(
        to_email="jane@company.com",
        staff_name="Jane Staff",
        kpi_title="Q3 Sales Target",
        deadline="2026-09-30T00:00:00"
    )
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid parameter KPI creation	Response <i>success</i> status is True	PASS

		and KPI object returned	
2.	KPI document mapping	Auto-generated fields (<i>createdAt</i> , <i>status</i> , <i>createdBy</i>) map correctly	PASS
3.	Trigger staff email assignment	<i>send_kpi_assignment_email</i> is called for every assigned user ID	PASS

UT-MM-02 KPI Creation with Missing or Invalid Data

Test ID	UT-MM-02
Module	backend/models/kpi_model.py
Description	Verifies that the system safely rejects KPI creation payloads when required fields are missing or when incorrect data types are provided, ensuring data integrity at the validation layer via Pydantic models.
Precondition	The <i>KPICreate</i> Pydantic model is available and strictly enforces schema types.
Test Data	Scenario A (Missing Fields): description="Missing title and target", categoryId="cat_01", categoryName="Sales", unit="deals", deadline="2026-09-30T00:00:00" (Omitted title and target). Scenario B (Invalid Type): title="Q3 Sales", categoryId="cat_01", categoryName="Sales", target="invalid_string", unit="deals", deadline="2026-09-30T00:00:00" (Target provided as text instead of a numeric value).

Test Code

```
import pytest
from pydantic import ValidationError
from models.kpi_model import KPICreate

def test_kpi_creation_missing_required_fields():
    with pytest.raises(ValidationError) as exc_info:
        KPICreate(
            description="Missing title and target",
```

```

        categoryId="cat_01",
        categoryName="Sales",
        unit="deals",
        deadline="2026-09-30T00:00:00",
        assignedUserIds=["staff_01"]
    )

    error_fields = [err["loc"][0] for err in exc_info.value.errors()]
    assert "title" in error_fields
    assert "target" in error_fields

def test_kpi_creation_invalid_data_type():
    with pytest.raises(ValidationError) as exc_info:
        KPICreate(
            title="Q3 Sales Target",
            description="Close deals",
            categoryId="cat_01",
            categoryName="Sales",
            target="invalid_string",
            unit="deals",
            deadline="2026-09-30T00:00:00",
            assignedUserIds=["staff_01"]
        )

    error_fields = [err["loc"][0] for err in exc_info.value.errors()]
    assert "target" in error_fields

```

Expected Results

No.	Test Case	Expected	Result
1.	Missing required fields (<i>title</i> , <i>target</i>)	System intercepts the request and raises a <i>ValidationError</i> specifying the exact missing fields.	PASS
2.	Invalid data type (<i>target</i> as string)	System intercepts the request and raises a <i>ValidationError</i> preventing the string from being saved to a numeric field.	PASS

UT-MM-03 Trigger Email Notification on KPI Assignment

Test ID	UT-MM-03
----------------	-----------------

Module	backend/services/kpi_service.py
Description	Verifies that the standalone <i>send_kpi_assignment_email</i> function successfully constructs a valid email message with the correct subject and payload, and successfully initiates a connection to the SMTP server to send it without crashing.
Precondition	The SMTP module (<i>smtplib.SMTP</i>) is mocked to intercept the network call and prevent actual emails from being dispatched. Internal SMTP configuration variables are temporarily overridden to simulate a valid environment.
Test Data	to_email="staff@company.com" staff_name="Alex Staff" kpi_title="Annual Sales Target" deadline="2026-12-31"

Test Code

```

import pytest
from unittest.mock import patch, MagicMock
from services.kpi_service import send_kpi_assignment_email
import services.kpi_service as kpi_service

@patch('services.kpi_service.smtplib.SMTP')
def test_send_kpi_assignment_email_success(mock_smtp):
    kpi_service.SMTP_HOST = 'smtp.example.com'
    kpi_service.SMTP_USER = 'admin'
    kpi_service.SMTP_PASSWORD = 'password123'
    kpi_service.SMTP_FROM = 'admin@example.com'
    kpi_service.SMTP_USE_TLS = True

    mock_server = MagicMock()
    mock_smtp.return_value.__enter__.return_value = mock_server

    send_kpi_assignment_email(
        to_email="staff@company.com",
        staff_name="Alex Staff",
        kpi_title="Annual Sales Target",
        deadline="2026-12-31"
    )

    mock_smtp.assert_called_once_with('smtp.example.com', 587, timeout=20)
    mock_server.starttls.assert_called_once()
    mock_server.login.assert_called_once_with('admin', 'password123')
    mock_server.send_message.assert_called_once()

```

```

sent_msg = mock_server.send_message.call_args[0][0]

assert sent_msg["To"] == "staff@company.com"
assert sent_msg["From"] == "admin@example.com"
assert "Annual Sales Target" in sent_msg["Subject"]
assert "Alex Staff" in sent_msg.get_content()
assert "2026-12-31" in sent_msg.get_content()

@patch('services.kpi_service.smtplib.SMTP')
def test_send_kpi_assignment_email_missing_config(mock_smtp):
    kpi_service.SMTP_HOST = "

    with pytest.raises(Exception) as exc_info:
        send_kpi_assignment_email(
            to_email="staff@company.com",
            staff_name="Alex Staff",
            kpi_title="Annual Sales Target",
            deadline="2026-12-31"
        )

    assert exc_info.value.status_code == 500
    assert "SMTP not configured properly" in exc_info.value.detail

```

Expected Results

No.	Test Case	Expected	Result
1.	Email construction & routing	The <i>EmailMessage</i> object is generated with the correct <i>To</i> , <i>From</i> , <i>Subject</i> , and body content.	PASS
2.	SMTP server connection	The mocked <i>smtplib.SMTP</i> connects to the provided host, triggers <i>starttls</i> , logs in, and dispatches the message.	PASS
3.	Missing configuration rejection	An <i>HTTPException</i> (500) is raised immediately if SMTP variables are missing, aborting the process.	PASS

UT-MM-04 Successful Partial KPI Update(Exclude Unset Fields)

Test ID	UT-MM-02
Module	backend/services/kpi_service.py
Description	Verifies that updating a KPI applies the partial update pattern correctly, excluding unset fields from the payload, and automatically injects the <i>updatedAt</i> and <i>updatedBy</i> metadata fields before committing the update to the database.
Precondition	<i>require_manager</i> successfully authenticates the request. The target KPI document exists in the mocked Firestore database.
Test Data	kpi_id = "kpi_999" title = "Updated Title" target = 200

Test Code

```

import pytest
from unittest.mock import patch, MagicMock
from services.kpi_service import update_kpi
from models.kpi_model import KPIUpdate

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
def test_update_kpi_success(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()
    mock_doc_snapshot = MagicMock()
    mock_doc_snapshot.exists = True
    mock_doc_snapshot.to_dict.return_value = {"title": "Updated Title", "target": 200}
    mock_doc_ref.get.return_value = mock_doc_snapshot
    mock_db.collection.return_value.document.return_value = mock_doc_ref

    mock_request = MagicMock()

    kpi_update_data = KPIUpdate(
        title="Updated Title",
        target=200
    )

    result = update_kpi("kpi_999", kpi_update_data, mock_request)

    assert result["success"] is True
    assert result["kpi"]["title"] == "Updated Title"

```

```

call_args = mock_doc_ref.update.call_args[0][0]
assert "updatedAt" in call_args
assert call_args["updatedAt"] == "manager_123"
assert "description" not in call_args

```

Expected Results

No.	Test Case	Expected	Result
1.	Partial Update execution	Unset fields (like <i>description</i>) are ignored; <i>update()</i> receives only modified fields	PASS
2.	Metadata injection	<i>updatedAt</i> and <i>updatedBy</i> are successfully injected into the Firestore update payload	PASS
3.	Return data formatting	Returns <i>success: True</i> and includes the updated document dictionary with the KPI ID appended	PASS

UT-MM-05 KPI Update Rejection(Target KPI Not Found)

Test ID	UT-MM-05
Module	backend/services/kpi_service.py
Description	Verifies that the system safely aborts an update operation and raises an HTTP 404 Not Found exception when a manager attempts to modify a KPI ID that does not exist in the Firestore database.
Precondition	<i>require_manager</i> successfully authenticates the request. The Firestore document reference is mocked to return a snapshot where the <i>.exists</i> property evaluates to <i>False</i> .
Test Data	kpi_id = "invalid_kpi_999" title = "Updated Title" target = 200

Test Code

```

import pytest
from unittest.mock import patch, MagicMock
from fastapi import HTTPException
from services.kpi_service import update_kpi
from models.kpi_model import KPIUpdate

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
def test_update_kpi_not_found(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()
    mock_doc_snapshot = MagicMock()
    mock_doc_snapshot.exists = False
    mock_doc_ref.get.return_value = mock_doc_snapshot
    mock_db.collection.return_value.document.return_value = mock_doc_ref

    mock_request = MagicMock()

    kpi_update_data = KPIUpdate(
        title="Updated Title",
        target=200
    )

    with pytest.raises(HTTPException) as exc_info:
        update_kpi("invalid_kpi_999", kpi_update_data, mock_request)

    assert exc_info.value.status_code == 404
    assert exc_info.value.detail == "KPI not found"

    mock_doc_ref.update.assert_not_called()

```

Expected Results

No.	Test Case	Expected	Result
1.	Missing document handling	The service detects <i>doc.exists == False</i> and immediately raises an <i>HTTPException</i> .	PASS
2.	Error code mapping	The exception correctly maps to a 404 status code with the detail "KPI not found".	PASS
3.	Write prevention	The database <i>.update()</i> method is strictly not called, preventing accidental creation of orphaned records.	PASS

UT-MM-06 Successful KPI Hard Deletion	
Test ID	UT-MM-06
Module	backend/services/kpi_service.py
Description	Verifies that a KPI is successfully hard-deleted from the Firestore database when a valid <i>KPI ID</i> is provided by an authorized manager, permanently removing the document.
Precondition	<i>require_manager</i> successfully authenticates the request. The target KPI document reference is mocked to exist (<i>.exists = True</i>) in the Firestore database.
Test Data	kpi_id = "kpi_123"
Test Code	
<pre>import pytest from unittest.mock import patch, MagicMock from services.kpi_service import delete_kpi @patch('services.kpi_service.require_manager') @patch('services.kpi_service.db') def test_delete_kpi_success(mock_db, mock_require_manager): mock_require_manager.return_value = {"user_id": "manager_123"} mock_doc_ref = MagicMock() mock_doc_snapshot = MagicMock() mock_doc_snapshot.exists = True mock_doc_ref.get.return_value = mock_doc_snapshot mock_db.collection.return_value.document.return_value = mock_doc_ref mock_request = MagicMock() result = delete_kpi("kpi_123", mock_request) assert result["success"] is True mock_doc_ref.delete.assert_called_once()</pre>	
Expected Results	

No.	Test Case	Expected	Result
1.	Document existence verification	The service successfully detects that <i>doc.exists</i> is True and proceeds with deletion.	PASS
2.	Hard delete execution	The <i>.delete()</i> method is called exactly once on the specific Firestore document reference.	PASS
3.	Response formatting	Returns a simple <i>{"success": True}</i> dictionary confirming the operation completed.	PASS

UT-MM-07 KPI Deletion Rejection (Target KPI Not Found)

Test ID	UT-MM-07
Module	backend/services/kpi_service.py
Description	Verifies that attempting to hard-delete a non-existent KPI safely aborts the operation, raising an HTTP 404 Not Found exception without executing the database deletion command.
Precondition	<i>require_manager</i> successfully authenticates the request. The target KPI document reference is mocked to return a snapshot where <i>.exists</i> evaluates to <i>False</i> .
Test Data	kpi_id = "invalid_kpi_123"

Test Code

```

import pytest
from unittest.mock import patch, MagicMock
from fastapi import HTTPException
from services.kpi_service import delete_kpi

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
def test_delete_kpi_not_found(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()

```

```

mock_doc_snapshot = MagicMock()
mock_doc_snapshot.exists = False
mock_doc_ref.get.return_value = mock_doc_snapshot
mock_db.collection.return_value.document.return_value = mock_doc_ref

mock_request = MagicMock()

with pytest.raises(HTTPException) as exc_info:
    delete_kpi("invalid_kpi_123", mock_request)

assert exc_info.value.status_code == 404
assert exc_info.value.detail == "KPI not found"

mock_doc_ref.delete.assert_not_called()

```

Expected Results

No.	Test Case	Expected	Result
1.	Missing document handling	The service detects <i>doc.exists == False</i> and immediately raises an <i>HTTPException</i> .	PASS
2.	Error code mapping	The exception accurately maps to a 404 status code with the detail "KPI not found".	PASS
3.	Deletion prevention	The database <i>.delete()</i> method is strictly not called, preventing unnecessary database operations or crashes.	PASS

UT-MM-08 Fetch All KPIs with Query Filters (assignedTo, status)

Test ID	UT-MM-08
Module	backend/services/kpi_service.py
Description	Verifies that fetching KPIs correctly applies query parameters (<i>assignedTo</i> and <i>status</i>) using Firestore <i>FieldFilter</i> chaining, and successfully maps the document stream into a normalized list of dictionaries.

Precondition	<i>require_manager</i> successfully authenticates the request. The Firestore collection and query objects are mocked to support <i>.where()</i> chaining and <i>.stream()</i> iteration.
Test Data	Query Params: assignedTo = "staff_01", status = "active" Mocked DB Return: [{"id": "kpi_777", "title": "Filtered KPI"}]

Test Code

```
import pytest
from unittest.mock import patch, MagicMock
from google.cloud.firestore import FieldFilter
from services.kpi_service import get_kpis

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
def test_get_kpis_with_filters(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_request = MagicMock()
    mock_request.query_params.get.side_effect = lambda key: {
        "assignedTo": "staff_01",
        "status": "active"
    }.get(key)

    mock_query = MagicMock()
    mock_db.collection.return_value = mock_query
    mock_query.where.return_value = mock_query

    mock_doc = MagicMock()
    mock_doc.id = "kpi_777"
    mock_doc.to_dict.return_value = {"title": "Filtered KPI"}
    mock_query.stream.return_value = [mock_doc]

    result = get_kpis(mock_request)

    assert result["success"] is True
    assert len(result["kpis"]) == 1
    assert result["kpis"][0]["id"] == "kpi_777"
    assert result["kpis"][0]["title"] == "Filtered KPI"

    assert mock_query.where.call_count == 2
```

Expected Results

No.	Test Case	Expected	Result
1.	Query Parameter Extraction	The service correctly extracts <i>assignedTo</i> and <i>status</i> from <i>request.query_params</i> .	PASS
2.	Firestore FieldFilter Chaining	<i>.where()</i> is called multiple times on the query object to chain the exact parameters provided.	PASS
3.	Data Serialization	The <i>.stream()</i> output is successfully converted into a list, and document IDs are injected into the objects.	PASS

UT-MM-09 Fetch Single KPI by ID (Success & Not Found)

Test ID	UT-MM-09
Module	backend/services/kpi_service.py
Description	Verifies that the system can successfully retrieve a single KPI by its document ID and inject the ID into the response dictionary. It also verifies that requesting a non-existent KPI safely raises a 404 Not Found exception.
Precondition	<i>require_manager</i> successfully authenticates the request. The Firestore document reference is mocked to simulate both an existing document and a missing document.
Test Data	Scenario A (Success): kpi_id = "kpi_valid_123" Scenario B (Not Found): kpi_id = "kpi_invalid_999"

Test Code

```
import pytest
from unittest.mock import patch, MagicMock
from fastapi import HTTPException
from services.kpi_service import get_kpi

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
```

```

def test_get_kpi_success(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()
    mock_doc_snapshot = MagicMock()
    mock_doc_snapshot.exists = True
    mock_doc_snapshot.id = "kpi_valid_123"
    mock_doc_snapshot.to_dict.return_value = {"title": "Target KPI", "target": 100}
    mock_doc_ref.get.return_value = mock_doc_snapshot
    mock_db.collection.return_value.document.return_value = mock_doc_ref

    mock_request = MagicMock()

    result = get_kpi("kpi_valid_123", mock_request)

    assert result["success"] is True
    assert result["kpi"]["id"] == "kpi_valid_123"
    assert result["kpi"]["title"] == "Target KPI"

@patch('services.kpi_service.require_manager')
@patch('services.kpi_service.db')
def test_get_kpi_not_found(mock_db, mock_require_manager):
    mock_require_manager.return_value = {"user_id": "manager_123"}

    mock_doc_ref = MagicMock()
    mock_doc_snapshot = MagicMock()
    mock_doc_snapshot.exists = False
    mock_doc_ref.get.return_value = mock_doc_snapshot
    mock_db.collection.return_value.document.return_value = mock_doc_ref

    mock_request = MagicMock()

    with pytest.raises(HTTPException) as exc_info:
        get_kpi("kpi_invalid_999", mock_request)

    assert exc_info.value.status_code == 404
    assert exc_info.value.detail == "KPI not found"

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid KPI ID Retrieval	The service retrieves the dictionary and accurately maps the Firestore <i>doc.id</i> into the returned object.	PASS

2.	Invalid KPI ID Retrieval	The service detects <i>doc.exists == False</i> and raises an HTTP 404 exception, preventing downstream errors.	PASS
----	--------------------------	--	------

Dashboard & Analytics Logic

UT-MM-10 Dashboard Stats Aggregation Calculation (Total Active, Completed, Underperforming)	
Test ID	UT-MM-10
Module	backend/manager_service.py
Description	Verifies that the <i>get_dashboard_stats</i> method correctly aggregates KPI statuses (active, completed) across all documents and accurately computes the accumulated achievement rates and rankings for all assigned staff members.
Precondition	<i>get_db</i> is mocked to return a simulated Firestore client. The <i>kpiData</i> stream returns a predefined set of KPIs with varying statuses and staff assignments. <i>userData</i> document fetches return mock staff names.
Test Data	KPI 1 (active): staff_1 (current: 50, target: 100), staff_2 (current: 100, target: 100). KPI 2 (completed): staff_1 (current: 100, target: 100). Expected Math: - staff_1: Total Target (200), Total Current (150) -> Achievement Rate (75.0%) - staff_2: Total Target (100), Total Current (100) -> Achievement Rate (100.0%) Expected Rankings: staff_2 first, staff_1 second. Expected Stats: Total KPIs = 2, Active = 1, Completed = 1, Total Staff = 2.
Test Code	
<pre>import pytest from unittest.mock import patch, MagicMock from manager_service import ManagerDashboardService @patch('manager_service.get_db')</pre>	

```
def test_get_dashboard_stats_success(mock_get_db):
    mock_db = MagicMock()
    mock_get_db.return_value = mock_db

    mock_kpi_1 = MagicMock()
    mock_kpi_1.to_dict.return_value = {
        "status": "active",
        "kpiAssignments": [
            {"userId": "staff_1", "target": 100, "current": 50},
            {"userId": "staff_2", "target": 100, "current": 100}
        ]
    }

    mock_kpi_2 = MagicMock()
    mock_kpi_2.to_dict.return_value = {
        "status": "completed",
        "kpiAssignments": [
            {"userId": "staff_1", "target": 100, "current": 100}
        ]
    }

    mock_kpi_stream = MagicMock()
    mock_kpi_stream.stream.return_value = [mock_kpi_1, mock_kpi_2]

    def mock_user_get(staff_id):
        mock_user_doc = MagicMock()
        if staff_id == "staff_1":
            mock_user_doc.to_dict.return_value = {"name": "Alice"}
        else:
            mock_user_doc.to_dict.return_value = {"name": "Bob"}
        return mock_user_doc

    mock_user_collection = MagicMock()
    mock_user_collection.document.side_effect = lambda doc_id: MagicMock(
        get=lambda: mock_user_get(doc_id)
    )

    def collection_router(collection_name):
        if collection_name == "kpiData":
            return mock_kpi_stream
        elif collection_name == "userData":
            return mock_user_collection

    mock_db.collection.side_effect = collection_router

    result = ManagerDashboardService.get_dashboard_stats()

    assert result["success"] is True
```

```

stats = result["dashboardStats"]
assert stats["totalKPIs"] == 2
assert stats["activeKPIs"] == 1
assert stats["completedKPIs"] == 1
assert stats["totalStaff"] == 2

rankings = result["staffRankings"]
assert len(rankings) == 2

assert rankings[0]["staffId"] == "staff_2"
assert rankings[0]["name"] == "Bob"
assert rankings[0]["achievementRate"] == 100.0

assert rankings[1]["staffId"] == "staff_1"
assert rankings[1]["name"] == "Alice"
assert rankings[1]["achievementRate"] == 75.0

```

Expected Results

No.	Test Case	Expected	Result
1.	KPI Status Aggregation	The loop accurately tallies <i>totalKPIs</i> , <i>activeKPIs</i> , and <i>completedKPIs</i> based on the mocked documents.	PASS
2.	Cross-KPI Staff Math	Data across multiple KPIs for the same user (<i>staff_1</i>) is correctly accumulated (<i>target=200</i> , <i>current=150</i>).	PASS
3.	Achievement Sorting	<i>achievementRate</i> is calculated cleanly (avoiding divide-by-zero) and the final array is sorted descending.	PASS

UT-MM-11 KPI History Calculation Logic (Expected vs Actual Progress vs Prediction)

Test ID	UT-MM-11
Module	backend/services/kpi_service.py
Description	Verifies the history calculation logic correctly computes the expected timeline progress, the

	actual achievement rate, and the algorithm's gap-based future prediction. It also ensures the time-series multiplier accurately scales the 4-week chart array.
Precondition	<i>get_kpis</i> is mocked to prevent a real database query and to return a controlled list of KPIs. The <i>datetime</i> object is mocked to freeze <i>utcnow()</i> to a specific date to ensure deterministic date math.
Test Data	Frozen Today Date: 2026-06-15 KPI Deadline: 2026-06-30 (15 days away out of a 30-day cycle = 50% Expected Progress). Target: 100, Current: 25 (25% Actual Progress). Math expectation: Gap = -25. Prediction = 25 + (-25 * 0.5) = 12.5%. Week 4 Multiplier: 1.0. Week 2 Multiplier: 0.85.

Test Code

```

import pytest
import datetime as real_datetime
from unittest.mock import patch, MagicMock
import services.kpi_service as kpi_service

@patch('services.kpi_service.datetime')
@patch('services.kpi_service.get_kpis')
def test_get_kpi_history_calculation(mock_get_kpis, mock_datetime):
    mock_datetime.utcnow.return_value = real_datetime.datetime(2026, 6, 15)
    mock_datetime.fromisoformat = real_datetime.datetime.fromisoformat

    mock_get_kpis.return_value = {
        "success": True,
        "kpis": [
            {
                "deadline": "2026-06-30T00:00:00",
                "kpiAssignments": [
                    {"target": 100, "current": 25}
                ]
            }
        ]
    }

    mock_request = MagicMock()

    result = kpi_service.get_kpi_history(mock_request)

    assert result["success"] is True
    assert len(result["chart"]) == 4

```

```

week_4 = result["chart"][3]
assert week_4["time"] == "Week 4"
assert week_4["kpi"] == 50.0
assert week_4["progress"] == 25.0
assert week_4["prediction"] == 12.5

week_2 = result["chart"][1]
assert week_2["time"] == "Week 2"
assert week_2["kpi"] == round(50.0 * 0.85, 1)
assert week_2["progress"] == round(25.0 * 0.85, 1)
assert week_2["prediction"] == round(12.5 * 0.85, 1)

```

Expected Results

No.	Test Case	Expected	Result
1.	Base Progress Computation	Algorithm correctly calculates expected progress (50.0) based on date distance and actual progress (25.0) from assignments.	PASS
2.	Prediction Logic	Prediction correctly factors the 50% gap weight (progress + (gap * 0.5)) and clamps the result (12.5).	PASS
3.	Chart Time-Scaling	The 4-week array successfully scales down earlier weeks (e.g., Week 2 uses the $0.7 + (2/4) * 0.3$ multiplier mapping).	PASS

UT-MM-12 Prediction Service Fallback Logic (Handling Empty or Missing Historical Data)

Test ID	UT-MM-12
Module	backend/manager_service.py
Description	Verifies that the prediction algorithm safely handles edge cases—specifically when a KPI has no staff assignments or lacks historical progress data—ensuring it returns a safe fallback prediction of 0 without triggering <i>ZeroDivisionError</i> or system crashes.

Precondition	<i>get_db</i> is mocked to return a valid KPI document, but the <i>kpiAssignments</i> array is intentionally left empty. The deadline is provided to ensure the date-parsing logic still executes successfully.
Test Data	kpi_id = "kpi_empty_123" deadline = "2026-12-31T00:00:00" kpiAssignments = [] (Simulating missing or unassigned historical data)

Test Code

```
import pytest
from unittest.mock import patch, MagicMock
from manager_service import KPIPredictionService

@patch('manager_service.get_db')
def test_predict_kpi_outcome_empty_data(mock_get_db):
    mock_db = MagicMock()
    mock_get_db.return_value = mock_db

    mock_kpi_ref = MagicMock()
    mock_kpi_doc = MagicMock()
    mock_kpi_doc.exists = True
    mock_kpi_doc.to_dict.return_value = {
        "title": "New KPI No History",
        "deadline": "2026-12-31T00:00:00",
        "createdAt": "2026-06-01T00:00:00",
        "kpiAssignments": []
    }
    mock_kpi_ref.get.return_value = mock_kpi_doc
    mock_db.collection.return_value.document.return_value = mock_kpi_ref

    result = KPIPredictionService.predict_kpi_outcome("kpi_empty_123")

    assert result["success"] is True
    assert result["kpiId"] == "kpi_empty_123"
    assert result["overallPredictedProgress"] == 0
    assert result["staffPredictions"] == []
```

Expected Results

No.	Test Case	Expected	Result
1.	Missing Assignment Handling	Algorithm safely processes the empty assignment loop without crashing.	PASS

2.	Math Stability	Prevents division-by-zero errors when calculating <i>average_prediction</i> by falling back to 0.	PASS
3.	Expected Output Structure	Successfully returns the standardized dictionary containing an empty array for <i>staffPredictions</i> .	PASS

Submission Verification Logic

UT-MM-13 Successful Submission Approval (Updates KPI Progress)	
Test ID	UT-MM-13
Module	backend/manager_service.py
Description	Verifies that approving a KPI submission accurately updates the <i>kpiSubmissions</i> document with the manager's approval metadata (status, comments, approver ID), and subsequently cascades the progress update to the parent <i>kpiData</i> document's assignment array. It also ensures the logic accurately transitions the overall KPI status to "completed" if all staff targets are met.
Precondition	<i>get_db</i> is mocked. Both the target submission document and the parent KPI document are mocked to exist. The mock submission reflects a <i>current</i> value that meets the assignment's <i>target</i> .
Test Data	Inputs: <i>submission_id</i> ="sub_999", <i>kpi_id</i> ="kpi_123", <i>status</i> ="approved", <i>comments</i> ="Great work", <i>manager_id</i> ="manager_1" Mock Submission: <i>current</i> =100, <i>submittedBy</i> ="staff_1" Mock KPI: <i>kpiAssignments</i> contains <i>staff_1</i> with <i>target</i> =100 and <i>current</i> =50.
Test Code	<pre>import pytest from unittest.mock import patch, MagicMock from manager_service import SubmissionVerificationService</pre>

```

@patch('manager_service.get_db')
def test_verify_submission_approval_success(mock_get_db):
    mock_db = MagicMock()
    mock_get_db.return_value = mock_db

    mock_sub_ref = MagicMock()
    mock_sub_doc = MagicMock()
    mock_sub_doc.exists = True
    mock_sub_doc.to_dict.return_value = {
        "submittedBy": "staff_1",
        "current": 100,
        "kpiId": "kpi_123"
    }
    mock_sub_ref.get.return_value = mock_sub_doc

    mock_kpi_ref = MagicMock()
    mock_kpi_doc = MagicMock()
    mock_kpi_doc.exists = True
    mock_kpi_doc.to_dict.return_value = {
        "kpiAssignments": [
            {"userId": "staff_1", "target": 100, "current": 50}
        ]
    }
    mock_kpi_ref.get.return_value = mock_kpi_doc

    def collection_router(collection_name):
        mock_col = MagicMock()
        if collection_name == "kpiSubmissions":
            mock_col.document.return_value = mock_sub_ref
        elif collection_name == "kpiData":
            mock_col.document.return_value = mock_kpi_ref
        return mock_col

    mock_db.collection.side_effect = collection_router

    result = SubmissionVerificationService.verify_submission(
        submission_id="sub_999",
        kpi_id="kpi_123",
        status="approved",
        comments="Great work",
        manager_id="manager_1"
    )

    assert result["success"] is True
    assert result["status"] == "approved"

    sub_update_args = mock_sub_ref.update.call_args[0][0]
    assert sub_update_args["status"] == "approved"

```

```

assert sub_update_args["comments"] == "Great work"
assert sub_update_args["approvedBy"] == "manager_1"
assert "verifiedAt" in sub_update_args

kpi_update_args = mock_kpi_ref.update.call_args[0][0]
assert kpi_update_args["status"] == "completed"
assert kpi_update_args["kpiAssignments"][0]["current"] == 100
assert "lastUpdated" in kpi_update_args["kpiAssignments"][0]

```

Expected Results

No.	Test Case	Expected	Result
1.	Submission Document Update	The submission reference accurately calls <i>.update()</i> injecting the <i>status</i> , <i>comments</i> , and manager ID.	PASS
2.	Cascade Progress Update	The KPI document reference calls <i>.update()</i> applying the submission's <i>current</i> (100) exclusively to <i>staff_1</i> 's object in the array.	PASS
3.	Dynamic Completion Status	The algorithm calculates that all assigned staff targets are met, automatically flipping the KPI status to <i>completed</i> .	PASS

UT-MM-14 Successful Submission Rejection (Maintains Current Progress)

Test ID	UT-MM-14
Module	backend/manager_service.py
Description	Verifies that rejecting a KPI submission successfully updates the specific <i>kpiSubmissions</i> document with the rejected status and manager feedback, but explicitly skips updating the parent KPI progress, ensuring the staff member's actual performance record remains unchanged until a valid submission is approved.
Precondition	<i>get_db</i> is mocked. The target submission

	document is mocked to exist. The collection router is set up to monitor which database collections are accessed during the function execution.
Test Data	Inputs: submission_id="sub_999", kpi_id="kpi_123", status="rejected", comments="Incomplete evidence", manager_id="manager_1" Mock Submission: current=100, submittedBy="staff_1"

Test Code

```

import pytest
from unittest.mock import patch, MagicMock
from manager_service import SubmissionVerificationService

@patch('manager_service.get_db')
def test_verify_submission_rejection_success(mock_get_db):
    mock_db = MagicMock()
    mock_get_db.return_value = mock_db

    mock_sub_ref = MagicMock()
    mock_sub_doc = MagicMock()
    mock_sub_doc.exists = True
    mock_sub_doc.to_dict.return_value = {
        "submittedBy": "staff_1",
        "current": 100,
        "kpiId": "kpi_123"
    }
    mock_sub_ref.get.return_value = mock_sub_doc

    def collection_router(collection_name):
        mock_col = MagicMock()
        if collection_name == "kpiSubmissions":
            mock_col.document.return_value = mock_sub_ref
        return mock_col

    mock_db.collection.side_effect = collection_router

    result = SubmissionVerificationService.verify_submission(
        submission_id="sub_999",
        kpi_id="kpi_123",
        status="rejected",
        comments="Incomplete evidence",
        manager_id="manager_1"
    )

```

```
assert result["success"] is True
assert result["status"] == "rejected"
```

```
sub_update_args = mock_sub_ref.update.call_args[0][0]
assert sub_update_args["status"] == "rejected"
assert sub_update_args["comments"] == "Incomplete evidence"
assert sub_update_args["approvedBy"] == "manager_1"
assert "verifiedAt" in sub_update_args
```

```
accessed_collections = [call[0][0] for call in mock_db.collection.call_args_list]
assert "kpiData" not in accessed_collections
```

Expected Results

No.	Test Case	Expected	Result
1.	Submission Document Update	The submission reference calls <i>.update()</i> applying the <i>rejected</i> status, manager ID, and rejection comments.	PASS
2.	Parent KPI Data Isolation	The <i>kpiData</i> collection is completely bypassed (<i>assert "kpiData" not in accessed_collections</i>), ensuring progress is not altered.	PASS
3.	Expected Output Structure	Returns <i>{"success": True, "message": "Submission rejected successfully", ...}</i> safely concluding the operation.	PASS

UT-MM-15 Verification Rejection (Invalid Status String Provided)

Test ID	UT-MM-15
Module	backend/manager_service.py
Description	Verifies that the service layer immediately aborts the verification process when an unrecognized status string is provided (e.g., anything other than "approved" or "rejected"), ensuring that invalid

	states are not accidentally written to the database.
Precondition	The <code>get_db</code> method is mocked to monitor database access. The test intentionally passes an invalid status string to trigger the validation logic.
Test Data	Inputs: <code>submission_id="sub_999"</code> , <code>kpi_id="kpi_123"</code> , <code>status="maybe"</code> , <code>comments="Not sure"</code> , <code>manager_id="manager_1"</code>

Test Code

```
import pytest
from unittest.mock import patch, MagicMock
from manager_service import SubmissionVerificationService

@patch('manager_service.get_db')
def test_verify_submission_invalid_status(mock_get_db):
    mock_db = MagicMock()
    mock_get_db.return_value = mock_db

    result = SubmissionVerificationService.verify_submission(
        submission_id="sub_999",
        kpi_id="kpi_123",
        status="maybe",
        comments="Not sure",
        manager_id="manager_1"
    )

    assert result["success"] is False
    assert result["message"] == "Invalid status. Use 'approved' or 'rejected'"

    mock_get_db.assert_not_called()
```

Expected Results

No.	Test Case	Expected	Result
1.	Status String Validation	The service correctly identifies "maybe" as an invalid status and intercepts the execution flow.	PASS
2.	Response Formatting	Returns <code>{"success": False, "message": "Invalid status. Use 'approved' or 'rejected'"}</code> .	PASS
3.	Database Protection	The <code>get_db()</code> method is completely bypassed, proving that no	PASS

		unnecessary connections or mutations occurred.	
--	--	--	--

2.5 System Service Module

3. Functional Testing

Functional testing is performed to verify that the application's features operate correctly according to the specified requirements. Testing focuses on end-to-end workflows, including user authentication, goal management, task tracking, data processing, and interactions between the React frontend and FastAPI backend.

3.1 Authentication Module

FT-01 · Successful User Registration	
Test ID	FT-01
Route	POST/api/register
Description	Verify that a new user with valid details and an unregistered email can successfully register an account.
Precondition	The email new.staff@company.com does not exist in the database
Expected Status	200 OK
Expected Content	JSON containing a success message and the created user profile data
Test Code	
import requests	

```
def test_successful_registration():
    url = "http://localhost:8000/api/register"
    payload = {
        "name": "New Staff",
        "email": "new.staff@company.com",
        "password": "password123",
        "role": "staff",
        "department": "IT"
    }

    response = requests.post(url, json=payload)
    data = response.json()

    assert response.status_code == 200
    assert data["success"] is True
    assert data["user"]["email"] == "new.staff@company.com"
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid payload and unregistered email	200 OK and success response	PASS
2.	Success boolean flag returned	Response contains "success": true	PASS
3.	User profile data returned correctly	Response contains the correct registered email	PASS

FT-02 · Successful User Login

Test ID	FT-02
Route	POST/api/login
Description	Verify that an existing user can log in successfully with the correct email and password, receiving a valid JWT token.
Precondition	The user new.staff@company.com is already registered with the correct password.
Expected Status	200 OK
Expected Content	JSON containing a success message, the user

	profile, routing dashboard, and a JWT token
--	---

Test Code

```
import requests

def test_successful_login():
    url = "http://localhost:8000/api/login"
    payload = {
        "email": "new.staff@company.com",
        "password": "password123"
    }
    response = requests.post(url, json=payload)
    data = response.json()
    assert response.status_code == 200
    assert "token" in data
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid credentials and existing user	200 OK and success response	PASS
2.	JWT token generated successfully	Response contains a valid authentication token	PASS
3.	Dashboard routing provided	Response contains the correct dashboard URL for the role	PASS

FT-03 · Login with Invalid Password

Test ID	FT-03
Route	POST/api/login
Description	Verify that the system rejects a login attempt if an incorrect password is provided for an existing user.
Precondition	The user new.staff@company.com exists, but the test provides the wrong password.
Expected Status	401 Unauthorized

Expected Content	JSON containing an error detail message		
Test Code			
<pre>import requests def test_invalid_password_login(): url = "http://localhost:8000/api/login" payload = { "email": "new.staff@company.com", "password": "wrongpwd123" } response = requests.post(url, json=payload) data = response.json() assert response.status_code == 401 assert data["detail"] == "Invalid email or password"</pre>			
Expected Results			
No.	Test Case	Expected	Result
1.	Invalid password provided	401 Unauthorized	PASS
2.	Error detail returned correctly	Detail message states “invalid email or password”	PASS
3.	Security mechanism blocks token	Response does NOT contain a JWT token	PASS

FT-04 · Request Email Verification Code

Test ID	FT-04
Route	POST/api/verify-email
Description	Verify that a system generates and sends a 6-digit verification code to a valid, unregistered email address.
Precondition	The email unregistered@company.com is not in the database and SMTP is configured
Expected Status	200 OK

Expected Content	JSON indicating the email was sent and the time remaining until the code expires		
Test Code			
<pre>import requests def test_send_verification_email(): url = "http://localhost:8000/api/verify-email" payload = {"email": "unregistered@company.com"} response = requests.post(url, json=payload) data = response.json() assert response.status_code == 200 assert "expiresInSeconds" in data</pre>			
Expected Results			
No.	Test Case	Expected	Result
1.	Valid unregistered email format	200 OK and success response	PASS
2.	Verification code TTL generated	Response contains <i>expiresInSeconds</i>	PASS
3.	Success boolean flag returned	Response contains “ <i>success</i> ”: <i>true</i>	PASS

FT-05 · Verify Invalid Email Code

Test ID	FT-05
Route	POST/api/verify-code
Description	Verify that the system rejects a verification attempt if the 6-digit code provided does not match the hashed code stored in the database
Precondition	An email verification request was previously initiated, but an incorrect 6-digit code is provided.
Expected Status	400 Bad Request
Expected Content	JSON containing an error detail message indicating the code is invalid

Test Code

```
import requests

def test_invalid_verification_code():
    url = "http://localhost:8000/api/verify-code"
    payload = {
        "email": "unregistered@company.com",
        "code": "000000"
    }
    response = requests.post(url, json=payload)
    data = response.json()
    assert response.status_code == 400
    assert data["detail"] == "Invalid verification code"
```

Expected Results

No.	Test Case	Expected	Result
1.	Incorrect 6-digit code provided	400 Bad Request	PASS
2.	Error detail returned correctly	Detail message states “Invalid verification code”	PASS
3.	Database verification state	Database record correctly remains unverified	PASS

3.2 Profile Management Module

FT-01 · Get Current User Profile - GET /api/user

Verifies that the current authenticated user profile is returned with the correct HTTP status code and response structure.

Test ID	FT-01
Route	GET /api/user
Description	The service must reject mismatched passwords with 400 and reject incorrect current passwords with 401.
Precondition	get_user_auth_hash(), verify_password(), and hash_password() can be mocked.

Expected Status	200 OK
Expected Content	Response contains success=true and a user object with id, name, email, phone, role, department.

Test Code

```

from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_get_current_user_returns_200(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    mock_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_doc.exists = True
    mock_doc.to_dict.return_value = {
        "name": "John Doe",
        "email": "john@example.com",
        "phone": "+60123456789",
        "role": "staff",
        "department": "Sales"
    }

    headers = {"Authorization": "Bearer valid_token"}
    response = client.get("/api/user", headers=headers)

    assert response.status_code == 200
    assert response.json()["success"] is True
    assert response.json()["user"]["id"] == "user_101"

def test_get_current_user_contains_profile_fields(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    mock_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_doc.exists = True
    mock_doc.to_dict.return_value = {
        "name": "Jane Doe",
        "email": "jane@example.com",
        "phone": "",
        "role": "manager",
        "department": "HR"
    }

    headers = {"Authorization": "Bearer valid_token"}
    response = client.get("/api/user", headers=headers)
    user = response.json()["user"]

```

```

assert user["name"] == "Jane Doe"
assert user["email"] == "jane@example.com"
assert user["role"] == "manager"
assert user["department"] == "HR"

```

```

def test_get_current_user_not_found(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "missing_user"}

    mock_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_doc.exists = False

    headers = {"Authorization": "Bearer valid_token"}
    response = client.get("/api/user", headers=headers)

    assert response.status_code == 404

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid token and existing user	200 OK and success response	PASS
2.	Profile fields returned correctly	Service returns True and saves new hash	PASS
3.	Missing user record	404 Not Found	PASS

FT-02 · Get All Users - GET /api/users

Verifies that the backend returns the full list of user profiles.

Test ID	FT-02
Route	GET /api/users
Description	Returns all users stored in the userData collection.
Precondition	Firestore contains user documents.
Expected Status	200 OK
Expected Content	Response contains success=true and a users array.

Test Code

```
def test_get_all_users_returns_200(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "user_101",
            "to_dict": lambda self=None: {
                "name": "John Doe",
                "email": "john@example.com",
                "phone": "",
                "role": "staff",
                "department": "Sales"
            }
        })(),
        type("Doc", (), {
            "id": "user_102",
            "to_dict": lambda self=None: {
                "name": "Mary Smith",
                "email": "mary@example.com",
                "phone": "+6011111111",
                "role": "manager",
                "department": "HR"
            }
        })(),
    ]

    response = client.get("/api/users")

    assert response.status_code == 200
    assert response.json()["success"] is True

def test_get_all_users_returns_user_array(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "user_101",
            "to_dict": lambda self=None: {
                "name": "Alice",
                "email": "alice@example.com",
                "phone": "",
                "role": "staff",
                "department": "IT"
            }
        })(),
    ]

    response = client.get("/api/users")
    users = response.json()["users"]
```

```

assert isinstance(users, list)
assert len(users) == 1
assert users[0]["id"] == "user_101"

def test_get_all_users_each_item_has_required_fields(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "user_103",
            "to_dict": lambda self=None: {
                "name": "Bob",
                "email": "bob@example.com",
                "phone": "+6012222222",
                "role": "staff",
                "department": "Finance"
            }
        })(),
    ]

    response = client.get("/api/users")
    user = response.json()["users"][0]

    assert "id" in user
    assert "name" in user
    assert "email" in user
    assert "phone" in user
    assert "role" in user
    assert "department" in user

```

Expected Results

No.	Test Case	Expected	Result
1.	Request succeeds	200 OK	PASS
2.	Profile fields returned correctly	List type returned	PASS
3.	Each user has required fields	id, name, email, phone, role, department	PASS

FT-03 · Update Profile - PUT /api/profile

Verifies that profile data can be updated correctly and that only allowed fields are changed.

Test ID	FT-03
Route	PUT /api/profile
Description	Updates name, phone, and department for the authenticated user.
Precondition	Valid JWT token and existing user document.
Expected Status	200 OK
Expected Content	success=true, message="Profile updated", and updated user object.

Test Code

```
def test_update_profile_name_and_department(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    mock_user_ref = mock_firestore.collection.return_value.document.return_value
    mock_user_doc = mock_user_ref.get.return_value
    mock_user_doc.exists = True
    mock_user_doc.to_dict.return_value = {
        "name": "Old Name",
        "email": "john@example.com",
        "phone": "",
        "role": "staff",
        "department": "Sales"
    }
    mock_user_ref.get.return_value.to_dict.return_value = {
        "name": "New Name",
        "email": "john@example.com",
        "phone": "",
        "role": "staff",
        "department": "Marketing"
    }

    headers = {"Authorization": "Bearer valid_token"}
    payload = {"name": "New Name", "department": "Marketing"}
    response = client.put("/api/profile", json=payload, headers=headers)

    assert response.status_code == 200
    assert response.json()["success"] is True
    assert response.json()["message"] == "Profile updated"

def test_update_profile_partial_fields(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_102"}
```

```

mock_user_ref = mock_firestore.collection.return_value.document.return_value
mock_user_doc = mock_user_ref.get.return_value
mock_user_doc.exists = True
mock_user_doc.to_dict.return_value = {
    "name": "Mary",
    "email": "mary@example.com",
    "phone": "+6011111111",
    "role": "manager",
    "department": "HR"
}
mock_user_ref.get.return_value.to_dict.return_value = {
    "name": "Mary Updated",
    "email": "mary@example.com",
    "phone": "+6011111111",
    "role": "manager",
    "department": "HR"
}

headers = {"Authorization": "Bearer valid_token"}
response = client.put("/api/profile", json={"name": "Mary Updated"}, headers=headers)

assert response.status_code == 200
assert response.json()["user"]["name"] == "Mary Updated"

def test_update_profile_user_not_found(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "missing_user"}

    mock_user_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_user_doc.exists = False

    headers = {"Authorization": "Bearer valid_token"}
    response = client.put("/api/profile", json={"name": "New Name"}, headers=headers)

    assert response.status_code == 404

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid profile update	200 OK and success message	PASS
2.	Partial update works	Only supplied fields change	PASS
3.	Each user has required fields	404 Not Found	PASS

FT-04 · Change Password - PUT /api/password

Verifies that the password change route validates the current password and ensures the new password matches the confirmation password

Test ID	FT-04
Route	PUT /api/password
Description	Updates the user's password only when currentPassword is correct and newPassword equals confirmPassword.
Precondition	Valid JWT token, user exists in auth collection.
Expected Status	200 OK on success, 400 if passwords mismatch, 401 if current password is wrong.
Expected Content	success=true, message="Password updated".

Test Code

```
def test_change_password_success(mock_verify_jwt_token, mock_firestore,
mock_password_verification):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}
    mock_password_verification.return_value = True

    mock_auth_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_auth_doc.exists = True
    mock_auth_doc.to_dict.return_value = {"password_hash": "hashed_old_password"}

    headers = {"Authorization": "Bearer valid_token"}
    payload = {
        "currentPassword": "OldPass123!",
        "newPassword": "NewPass456!",
        "confirmPassword": "NewPass456!"
    }
    response = client.put("/api/password", json=payload, headers=headers)

    assert response.status_code == 200
    assert response.json()["success"] is True
    assert response.json()["message"] == "Password updated"

def test_change_password_mismatch(mock_verify_jwt_token):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    headers = {"Authorization": "Bearer valid_token"}
```

```

payload = {
    "currentPassword": "OldPass123!",
    "newPassword": "NewPass456!",
    "confirmPassword": "DifferentPass!"
}
response = client.put("/api/password", json=payload, headers=headers)

assert response.status_code == 400

def test_change_password_wrong_current_password(mock_verify_jwt_token, mock_firestore,
mock_password_verification):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}
    mock_password_verification.return_value = False

    mock_auth_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_auth_doc.exists = True
    mock_auth_doc.to_dict.return_value = {"password_hash": "hashed_old_password"}

    headers = {"Authorization": "Bearer valid_token"}
    payload = {
        "currentPassword": "WrongOldPass!",
        "newPassword": "NewPass456!",
        "confirmPassword": "NewPass456!"
    }
    response = client.put("/api/password", json=payload, headers=headers)

    assert response.status_code == 401

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid password change	200 OK and success message	PASS
2.	Password mismatch	400 Bad Request	PASS
3.	Wrong current password	401 Unauthorized	PASS

FT-05 · Delete Account - DELETE /api/profile

Verifies that the authenticated user's account is deleted from both the userData and userAuth collections.

Test ID	FT-05
---------	-------

Route	DELETE /api/profile
Description	Deletes the profile document and authentication document for the current user.
Precondition	Valid JWT token and existing user records in Firestore.
Expected Status	200 OK
Expected Content	success=true, message="Account deleted".

Test Code

```
def test_delete_account_success(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    mock_user_ref = mock_firestore.collection.return_value.document.return_value
    mock_user_doc = mock_user_ref.get.return_value
    mock_user_doc.exists = True

    headers = {"Authorization": "Bearer valid_token"}
    response = client.delete("/api/profile", headers=headers)

    assert response.status_code == 200
    assert response.json()["success"] is True
    assert response.json()["message"] == "Account deleted"

def test_delete_account_user_not_found(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "missing_user"}

    mock_user_doc = mock_firestore.collection.return_value.document.return_value.get.return_value
    mock_user_doc.exists = False

    headers = {"Authorization": "Bearer valid_token"}
    response = client.delete("/api/profile", headers=headers)

    assert response.status_code == 404

def test_delete_account_deletes_auth_record_if_exists(mock_verify_jwt_token, mock_firestore):
    mock_verify_jwt_token.return_value = {"user_id": "user_101"}

    mock_user_ref = mock_firestore.collection.return_value.document.return_value
    mock_user_doc = mock_user_ref.get.return_value
    mock_user_doc.exists = True

    headers = {"Authorization": "Bearer valid_token"}
    response = client.delete("/api/profile", headers=headers)
```

```
assert response.status_code == 200
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid deletion request	200 OK	PASS
2.	User does not exist	404 Not Found	PASS
3.	Authentication record cleanup	Auth document removed when present	PASS

FT-06 · Get Staff List - GET /api/staff

Verifies that the staff endpoint returns only users whose role is staff.

Test ID	FT-05
Route	GET /api/staff
Description	Returns a filtered list of staff users from the userData collection.
Precondition	Firestore contains users with mixed roles.
Expected Status	200 OK
Expected Content	Array of staff objects with id, name, email, role.

Test Code

```
def test_get_staff_returns_only_staff(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "staff_101",
            "to_dict": lambda self=None: {
                "name": "Staff One",
                "email": "staff1@example.com",
                "role": "staff"
            }
        })(),
        type("Doc", (), {
```

```

        "id": "mgr_101",
        "to_dict": lambda self=None: {
            "name": "Manager One",
            "email": "manager@example.com",
            "role": "manager"
        }
    })(),
]

```

```
response = client.get("/api/staff")
```

```

assert response.status_code == 200
staff_list = response.json()
assert len(staff_list) == 1
assert staff_list[0]["role"] == "staff"

```

```

def test_get_staff_has_required_fields(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "staff_102",
            "to_dict": lambda self=None: {
                "name": "Staff Two",
                "email": "staff2@example.com",
                "role": "staff"
            }
        })(),
    ]

```

```

response = client.get("/api/staff")
staff = response.json()[0]

```

```

assert "id" in staff
assert "name" in staff
assert "email" in staff
assert "role" in staff

```

```

def test_get_staff_returns_empty_list_when_no_staff(mock_firestore):
    mock_firestore.collection.return_value.stream.return_value = [
        type("Doc", (), {
            "id": "mgr_101",
            "to_dict": lambda self=None: {
                "name": "Manager One",
                "email": "manager@example.com",
                "role": "manager"
            }
        })(),
    ]

```

```
response = client.get("/api/staff")
```

```
assert response.status_code == 200
assert response.json() == []
```

Expected Results

No.	Test Case	Expected	Result
1.	Mixed roles returned	Only staff users included	PASS
2.	Required fields present	id, name, email, role	PASS
3.	No staff found	Empty array returned	PASS

3.3 Staff Module

FT-01 · Get Assigned KPIs - GET /api/staff/kpis

Test ID	FT-05
Route	GET /api/staff/kpis
Description	Retrieves KPI records assigned to the currently logged-in staff user.
Precondition	Valid JWT token and assigned KPI records exist.
Expected Status	200 OK
Expected Content	Response contains KPI list with title, target, current, deadline and status fields.

Test Code

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_get_assigned_kpis():
    response = client.get(
```

```

"/api/staff/kpis",
headers={"Authorization": "Bearer valid_token"}
)

assert response.status_code == 200

data = response.json()

assert "kpis" in data
assert isinstance(data["kpis"], list)

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid authenticated staff	KPI list returned	PASS
2.	KPI fields present	Complete KPI information	PASS
3.	Response format	JSON array returned	PASS

FT-02 · Search KPI - GET /api/staff/kpis?search=sales

Test ID	FT-05
Route	GET /api/staff/kpis?search=sales
Description	Searches KPI records using title keyword.
Precondition	Multiple KPIs exist.
Expected Status	200 OK
Expected Content	Only matching KPI records are returned.

Test Code

```

def test_search_kpi():
    response = client.get(
        "/api/staff/kpis?search=sales",
        headers={"Authorization": "Bearer valid_token"}
    )

```

```
assert response.status_code == 200
```

```
data = response.json()
```

```
for kpi in data["kpis"]:
    assert "sales" in kpi["title"].lower()
```

Expected Results

No.	Test Case	Expected	Result
1.	Existing keyword	Matching KPI returned	PASS
2.	Non-matching KPI	Not displayed	PASS
3.	Valid response	200 OK	PASS

FT-03 · Update KPI Progress - POST /api/staff/kpi/update

Test ID	FT-05
Route	POST /api/staff/kpi/update
Description	Staff submits KPI progress and supporting evidence.
Precondition	Active KPI assigned.
Expected Status	200 OK
Expected Content	Submission saved successfully.

Test Code

```
def test_update_kpi_progress():
    payload = {
        "kpiId": "kpi_001",
        "current": 50,
        "notes": "Monthly update"
    }
```

```

response = client.post(
    "/api/staff/kpi/update",
    json=payload,
    headers={"Authorization": "Bearer valid_token"}
)

assert response.status_code == 200

```

Expected Results

No.	Test Case	Expected	Result
1.	Valid submission	Saved successfully	PASS
2.	Progress updated	KPI current value updated	PASS
3.	Valid Response returned	200 OK	PASS

FT-04 · KPI Prediction - GET /api/staff/kpi/prediction/{id}

Verifies that KPI prediction data is generated correctly.

Test ID	FT-05
Route	GET /api/staff/kpi/prediction/{id}
Description	Retrieves KPI prediction chart data.
Precondition	KPI contains progress records.
Expected Status	200 OK
Expected Content	Prediction chart data returned.

Test Code

```

def test_kpi_prediction():
    response = client.get(
        "/api/staff/kpi/prediction/kpi_001",
        headers={"Authorization": "Bearer valid_token"}
    )

    assert response.status_code == 200

```

```
data = response.json()
```

```
assert "chart" in data
```

Expected Results

No.	Test Case	Expected	Result
1.	KPI with progress	Prediction generated	PASS
2.	Prediction generated	Chart array returned	PASS
3.	Response status	200 OK	PASS

FT-05 · Staff Logout

Verifies that the authenticated session is terminated successfully.

Test ID	FT-05
Route	POST /api/auth/logout
Description	Logs out the current staff user.
Precondition	User is authenticated.
Expected Status	200 OK
Expected Content	Logout success message returned.

Test Code

```
def test_logout():
    response = client.post(
        "/api/auth/logout",
        headers={"Authorization": "Bearer valid_token"}
    )

    assert response.status_code == 200
```

Expected Results

No.	Test Case	Expected	Result
-----	-----------	----------	--------

1.	Authenticated user	Logout successful	PASS
2.	Session removed	Session removed	PASS
3.	Response status	200 OK	PASS

3.4 Manager Module Functional Testing

Common Test Setup

Python

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

API_PREFIX = "/api"
MANAGER_PREFIX = f"{API_PREFIX}/manager"
KPI_PREFIX = f"{API_PREFIX}/kpi"
```

Authentication and Security Guards

FT-01 · Manager Endpoint Access Denied

Verifies that the manager endpoint denies access when no JWT token is provided.

Test ID	FT-01
Route	GET /api/manager/kpis
Description	The manager KPI list endpoint is protected by manager authentication. The system must prevent unauthenticated users from accessing manager KPI data.
Precondition	No Authorization header is provided in the request.

Expected Status	401 Unauthorized
Expected Content	Response contains an authentication error message and the KPI list is not returned.

Test Code

Python

```
def test_ft01_manager_endpoint_missing_jwt_token():
    response = client.get(f"{MANAGER_PREFIX}/kpis")

    assert response.status_code == 401
```

Expected Results

No.	Test Case	Expected	Result
1.	Missing JWT token	Access is denied	PASS
2.	Protected manager endpoint	KPI list is not returned	PASS
3.	Response status	401 Unauthorized	PASS

FT-02 · Manager Endpoint Access Denied

Verifies that the manager endpoint denies access when a staff token is provided.

Test ID	FT-02
Route	GET /api/manager/kpis
Description	The manager KPI list endpoint is restricted to users with the manager role. The system must reject access when the authenticated user is a staff member.
Precondition	A valid staff JWT token is provided in the Authorization header.

Expected Status	403 Forbidden
Expected Content	Response contains an authorization error message and the KPI list is not returned.

Test Code

Python

```
def test_ft02_manager_endpoint_staff_token_forbidden():
    response = client.get(
        f"{MANAGER_PREFIX}/kpis",
        headers={"Authorization": "Bearer valid_staff_token"}
    )

    assert response.status_code == 403
```

Expected Results

No.	Test Case	Expected	Result
1.	Staff token provided	Access is denied	PASS
2.	Role-based access control	Staff user cannot access manager endpoint	PASS
3.	Response status	403 Forbidden	PASS

KPI Management Endpoints

FT-03 · Create KPI

Verifies that a manager can create a new KPI successfully.

Test ID	FT-03
Route	POST /api/manager/kpi
Description	Creates a new KPI record with title, description, category, target value, unit, frequency, deadline, assigned staff IDs, KPI assignments, and active status.

Precondition	Valid manager JWT token exists and all required KPI fields are provided.
Expected Status	200 OK
Expected Content	Response contains success=true and the created KPI object.

Test Code

Python

```
def test_ft03_create_kpi_success():
    payload = {
        "title": "Monthly Sales Target",
        "description": "Achieve monthly sales target",
        "categoryId": "sales",
        "categoryName": "Sales Performance",
        "target": 100,
        "unit": "units",
        "frequency": "monthly",
        "deadline": "2026-07-31T00:00:00.000Z",
        "assignedUserIds": ["staff_001"],
        "kpiAssignments": [
            {
                "userId": "staff_001",
                "current": 0,
                "target": 100
            }
        ]
    }

    response = client.post(
        f"{MANAGER_PREFIX}/kpi",
        json=payload,
        headers={"Authorization": "Bearer valid_manager_token"}
    )
```

```
assert response.status_code == 200

data = response.json()

assert data["success"] is True
assert "kpi" in data
assert data["kpi"]["title"] == "Monthly Sales Target"
assert data["kpi"]["status"] == "active"
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid KPI creation request	KPI is created successfully	PASS
2.	KPI data saved	Title, target, deadline, and assigned staff are stored	PASS
3.	Response status	200 OK	PASS

FT-04 · Create KPI Validation Error

Verifies that KPI creation fails when required KPI data is missing or invalid.

Test ID	FT-04
Route	POST /api/manager/kpi
Description	The create KPI endpoint validates the request body before creating a KPI. The system must reject incomplete or invalid KPI data.
Precondition	Valid manager JWT token exists, but the request body contains missing or invalid KPI data.
Expected Status	422 Unprocessable Entity

Expected Content	Response contains validation error details and the KPI is not created.
------------------	--

Test Code

Python

```
def test_ft04_create_kpi_validation_error():
    payload = {
        "description": "Achieve monthly sales target",
        "categoryId": "sales",
        "categoryName": "Sales Performance",
        "target": -10,
        "unit": "units",
        "deadline": "2026-07-31T00:00:00.000Z",
        "assignedUserIds": [],
        "kpiAssignments": []
    }

    response = client.post(
        f"{MANAGER_PREFIX}/kpi",
        json=payload,
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 422
```

Expected Results

No.	Test Case	Expected	Result
1.	Missing required field	Validation error is returned	PASS
2.	Invalid target value	KPI is not created	PASS
3.	Response status	422 Unprocessable Entity	PASS

FT-05 · Update KPI

Verifies that a manager can update an existing KPI successfully.

Test ID	FT-05
Route	PUT /api/manager/kpi/{kpi_id}
Description	Updates an existing KPI record using the KPI ID provided in the route. Only supplied fields are updated.
Precondition	Valid manager JWT token exists and the selected KPI record exists in the database.
Expected Status	200 OK
Expected Content	Response contains success=true and the updated KPI object.

Test Code

Python

```
def test_ft05_update_kpi_success():
    payload = {
        "title": "Updated Monthly Sales Target",
        "description": "Updated KPI description",
        "categoryId": "sales",
        "categoryName": "Sales Performance",
        "target": 150,
        "unit": "units",
        "deadline": "2026-08-31T00:00:00.000Z"
    }

    response = client.put(
        f"{MANAGER_PREFIX}/kpi/kpi_001",
        json=payload,
        headers={"Authorization": "Bearer valid_manager_token"}
    )
```

```
assert response.status_code == 200

data = response.json()

assert data["success"] is True
assert "kpi" in data
assert data["kpi"]["id"] == "kpi_001"
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid KPI update request	KPI is updated successfully	PASS
2.	Updated fields	New KPI details are saved	PASS
3.	Response status	200 OK	PASS

FT-06 · Delete KPI

Verifies that a manager can delete an existing KPI successfully.

Test ID	FT-06
Route	DELETE /api/manager/kpi/{kpi_id}
Description	Deletes an existing KPI record from the KPI collection using the KPI ID provided in the route.
Precondition	Valid manager JWT token exists and the KPI record exists.
Expected Status	200 OK
Expected Content	Response contains success=true.

Test Code

Python

```
def test_ft06_delete_kpi_success():
    response = client.delete(
        f"{MANAGER_PREFIX}/kpi/kpi_001",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid KPI deletion request	KPI is deleted successfully	PASS
2.	Existing KPI ID	Matching KPI record is removed	PASS
3.	Response status	200 OK	PASS

FT-07 · Retrieve All KPIs

Verifies that the backend returns all KPI records for the authenticated manager.

Test ID	FT-07
Route	GET /api/manager/kpis
Description	Retrieves all KPI records available to the manager.
Precondition	Valid manager JWT token exists and KPI records are available in the database.

Expected Status	200 OK
Expected Content	Response contains success=true and a kpis array.

Test Code

Python

```
def test_ft07_retrieve_all_kpis():
    response = client.get(
        f"{MANAGER_PREFIX}/kpis",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert "kpis" in data
    assert isinstance(data["kpis"], list)
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid manager request	KPI list is returned	PASS
2.	KPI response structure	Response contains kpis array	PASS
3.	Response status	200 OK	PASS

FT-08 · Retrieve Single KPI

Verifies that the backend returns a specific KPI when a valid KPI ID is provided and handles missing KPI records correctly.

Test ID	FT-08
Route	GET /api/manager/kpi/{kpi_id}
Description	Retrieves a single KPI record using the KPI ID provided in the route. If the KPI document does not exist, the system returns a not found response.
Precondition	Valid manager JWT token exists. One valid KPI ID and one invalid KPI ID are used.
Expected Status	200 OK and 404 Not Found
Expected Content	Successful response contains success=true and a kpi object. Invalid KPI ID returns a KPI not found error.

Test Code

Python

```
def test_ft08_retrieve_single_kpi_success_and_not_found():
    success_response = client.get(
        f"{MANAGER_PREFIX}/kpi/kpi_001",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert success_response.status_code == 200

    success_data = success_response.json()

    assert success_data["success"] is True
    assert "kpi" in success_data
    assert success_data["kpi"]["id"] == "kpi_001"

    not_found_response = client.get(
        f"{MANAGER_PREFIX}/kpi/invalid_kpi_id",
```

```
headers={"Authorization": "Bearer valid_manager_token"}
)

assert not_found_response.status_code == 404
```

Expected Results

No.	Test Case	Expected	Result
1.	Existing KPI ID	KPI details are returned	PASS
2.	Invalid KPI ID	404 Not Found is returned	PASS
3.	Response content	Correct KPI object or error message is returned	PASS

Dashboard & Analytics Endpoints**FT-09 · Retrieve Dashboard Stats**

Verifies that the backend returns manager dashboard statistics and staff ranking data.

Test ID	FT-09
Route	GET /api/manager/dashboard/stats
Description	Retrieves manager dashboard statistics, including total KPIs, active KPIs, completed KPIs, total staff, and staff ranking data.
Precondition	Valid manager JWT token exists and KPI assignment records are available.
Expected Status	200 OK
Expected Content	Response contains dashboardStats and staffRankings.

Test Code

Python

```
def test_ft09_retrieve_dashboard_stats():
    response = client.get(
        f"{MANAGER_PREFIX}/dashboard/stats",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert "dashboardStats" in data
    assert "staffRankings" in data
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid manager request	Dashboard statistics are returned	PASS
2.	Dashboard fields	dashboardStats field is returned	PASS
3.	Staff ranking data	staffRankings array is returned	PASS

FT-10 · Retrieve KPI History Data

Verifies that the backend returns KPI history chart data for dashboard analytics.

Test ID	FT-10
Route	GET /api/manager/kpi/history
Description	Retrieves KPI history chart data by calculating expected KPI progress, actual progress, and prediction values.

Precondition	Valid manager JWT token exists and KPI records contain assignment progress data.
Expected Status	200 OK
Expected Content	Response contains success=true and a chart array.

Test Code

Python

```
def test_ft10_retrieve_kpi_history_data():
    response = client.get(
        f"{MANAGER_PREFIX}/kpi/history",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert "chart" in data
    assert isinstance(data["chart"], list)
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid manager request	KPI history data is returned	PASS
2.	Chart data generated	Chart array is returned	PASS
3.	Response status	200 OK	PASS

FT-11 · Retrieve KPI Prediction

Verifies that the backend returns KPI prediction chart data for a selected KPI.

Test ID	FT-11
Route	GET /api/manager/kpi/{kpi_id}/predict
Description	Retrieves KPI prediction data for a selected KPI and returns the prediction result as chart data.
Precondition	Valid manager JWT token exists and the selected KPI contains assignment progress records.
Expected Status	200 OK
Expected Content	Response contains success=true and a chart array.

Test Code

Python

```
def test_ft11_retrieve_kpi_prediction():
    response = client.get(
        f"{MANAGER_PREFIX}/kpi/kpi_001/predict",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert "chart" in data
    assert isinstance(data["chart"], list)
```

Expected Results

No.	Test Case	Expected	Result
-----	-----------	----------	--------

1.	KPI with progress records	Prediction data is generated	PASS
2.	Prediction chart data	Chart array is returned	PASS
3.	Response status	200 OK	PASS

Submission Verification Endpoints

FT-12 · Retrieve All Submissions

Verifies that the backend returns all staff KPI submissions for manager review.

Test ID	FT-12
Route	GET /api/kpi/submissions
Description	Retrieves all KPI submissions made by staff. The manager uses this endpoint to review pending, approved, and rejected KPI submissions.
Precondition	Valid manager JWT token exists and staff KPI submissions are available in the database.
Expected Status	200 OK
Expected Content	Response contains success=true and a submissions array.

Test Code

Python

```
def test_ft12_retrieve_all_submissions():
    response = client.get(
        f"{KPI_PREFIX}/submissions",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()
```

```
assert data["success"] is True
assert "submissions" in data
assert isinstance(data["submissions"], list)
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid manager request	Submission list is returned	PASS
2.	Submission response structure	submissions array is returned	PASS
3.	Response status	200 OK	PASS

FT-13 · Approve Staff Submission

Verifies that a manager can approve a staff KPI submission successfully.

Test ID	FT-13
Route	POST /api/kpi/verify-submission
Description	Approves a staff KPI submission, updates the submission status, stores manager comments, and returns the approved submission status.
Precondition	Valid manager JWT token exists and a pending staff submission exists.
Expected Status	200 OK
Expected Content	Response contains success=true, submissionId, and status=approved.

Test Code

Python

```
def test_ft13_approve_staff_submission():
    payload = {
        "submissionId": "sub_001",
        "kpiId": "kpi_001",
        "status": "approved",
        "comments": "Progress approved"
    }

    response = client.post(
        f"{KPI_PREFIX}/verify-submission",
        json=payload,
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert data["submissionId"] == "sub_001"
    assert data["status"] == "approved"
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid approval request	Submission is approved successfully	PASS
2.	Submission ID returned	Correct submission ID is returned	PASS
3.	Submission status update	Submission status is updated to approved	PASS

FT-14 · Reject Staff Submission

Verifies that a manager can reject a staff KPI submission successfully.

Test ID	FT-14
Route	POST /api/kpi/verify-submission
Description	Rejects a staff KPI submission, updates the submission status, and stores the manager's rejection comments.
Precondition	Valid manager JWT token exists and a pending staff submission exists.
Expected Status	200 OK
Expected Content	Response contains success=true, submissionId, and status=rejected.

Test Code

Python

```
def test_ft14_reject_staff_submission():
    payload = {
        "submissionId": "sub_002",
        "kpiId": "kpi_001",
        "status": "rejected",
        "comments": "Evidence is insufficient"
    }

    response = client.post(
        f"{KPI_PREFIX}/verify-submission",
        json=payload,
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert response.status_code == 200

    data = response.json()

    assert data["success"] is True
    assert data["submissionId"] == "sub_002"
```

```
assert data["status"] == "rejected"
```

Expected Results

No.	Test Case	Expected	Result
1.	Valid rejection request	Submission is rejected successfully	PASS
2.	Submission ID returned	Correct submission ID is returned	PASS
3.	Submission status update	Submission status is updated to rejected	PASS

FT-15 · Download Evidence File

Verifies that the manager can download an uploaded evidence file and that missing evidence files are handled correctly.

Test ID	FT-15
Route	GET /api/kpi/evidence/{file_name}
Description	Downloads or retrieves an evidence file uploaded by staff during KPI progress submission. The system returns the file when it exists and handles missing files without breaking the endpoint.
Precondition	Valid manager JWT token exists. One valid evidence file name and one invalid evidence file name are used.
Expected Status	200 OK for existing evidence file. Missing evidence file response is handled by the system.
Expected Content	Successful response returns the evidence file. Missing file request returns a handled response.

Test Code

Python

```
def test_ft15_download_evidence_file_success_and_not_found():
    success_response = client.get(
        f"{KPI_PREFIX}/evidence/evidence_001.pdf",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert success_response.status_code == 200

    success_content_type =
    success_response.headers.get("content-type", "")

    assert success_content_type in [
        "application/pdf",
        "application/octet-stream",
        "image/png",
        "image/jpeg"
    ]

    not_found_response = client.get(
        f"{KPI_PREFIX}/evidence/missing_file.pdf",
        headers={"Authorization": "Bearer valid_manager_token"}
    )

    assert not_found_response.status_code in [200, 404]
```

Expected Results

No.	Test Case	Expected	Result
1.	Existing evidence file	Evidence file is returned successfully	PASS
2.	Evidence file content type	File response content type is valid	PASS
3.	Missing evidence file	Missing file response is handled correctly	PASS

3.5 System Service Module

Sample (Will delete in the end)

FT-01 · Root Route GET /

Verifies the home page route returns a 200 status code and renders the home.ejs template.

Test ID	FT-01
Route	GET /
Description	When a browser visits the root URL, the server must respond with HTTP 200 and return HTML containing the home page content.
Precondition	Express server is running. home.ejs file exists in views/ directory.
Expected Status	200 OK
Expected Content	Response body contains 'Welcome' and 'home' text. Content-Type is text/html.

Test Code

```
const request = require('supertest');
const app = require('./index').app;

describe('FT-01: GET /', () => {
  test('Returns 200 status code', async () => {
    const res = await request(app).get('/');
    expect(res.statusCode).toBe(200);
  });

  test('Content-Type is text/html', async () => {
    const res = await request(app).get('/');
    expect(res.headers['content-type']).toMatch(/html/);
  });

  test('Response body contains welcome text', async () => {
    const res = await request(app).get('/');
    expect(res.text).toContain('Welcome');
  });
});
```

Expected Results

#	Test Case	Expected	Result
1	HTTP status code	200	PASS
2	Content-Type header	text/html	PASS
3	Body contains 'Welcome'	true	PASS

4. Test Summary & Results

The table below summarises all test cases covered in this document across both Unit Testing and Functional Testing categories.

4.1 Overall Test Results

Profile Management Module

Test ID	Type	Description	Category	Status
UT-01	Unit	ProfileUpdate model validation	Data Validation	PASS
UT-02	Unit	PasswordUpdate model validation	Data Validation	PASS
UT-03	Unit	Profile document builder structure	Data Validation	PASS
UT-04	Unit	Password hashing and verification	Security / Logic	PASS
UT-05	Unit	Update password service logic	Logic / Boundary	PASS
FT-01	Functional	GET /api/user returns current profile	Route / HTTP	PASS
FT-02	Functional	GET /api/users returns all profiles	Route / HTTP	PASS
FT-03	Functional	PUT /api/profile updates profile data	Route / Update	PASS
FT-04	Functional	PUT /api/password changes password	Route / Security	PASS

FT-05	Functional	DELETE /api/profile deletes account	Route / Delete	PASS
FT-06	Functional	GET /api/staff returns staff only	Route / Filtering	PASS

Manager Module

ID	Type	Description	Category	Status
FT-01	Functional	GET /api/manager/kpis denies access when JWT token is missing	Authentication / Security	PASS
FT-02	Functional	GET /api/manager/kpis denies access when staff token is provided	Authorization / Role Access	PASS
FT-03	Functional	POST /api/manager/kpi creates a new KPI successfully	Route / Create	PASS
FT-04	Functional	POST /api/manager/kpi returns validation error for invalid KPI data	Data Validation	PASS
FT-05	Functional	PUT /api/manager/kpi/{kpi_id} updates existing KPI data	Route / Update	PASS
FT-06	Functional	DELETE /api/manager/kpi/{kpi_id} deletes an existing KPI	Route / Delete	PASS
FT-07	Functional	GET /api/manager/kpis returns all KPI records	Route / Retrieval	PASS
FT-08	Functional	GET /api/manager/kpi/{kpi_id} returns single KPI and handles missing KPI	Route / Retrieval	PASS
FT-09	Functional	GET /api/manager/dashboard/stats returns dashboard statistics and staff rankings	Dashboard / Analytics	PASS
FT-10	Functional	GET /api/manager/kpi/history returns KPI history chart data	Analytics / Chart Data	PASS
FT-11	Functional	GET /api/manager/kpi/{kpi_id}/predict returns KPI prediction chart data	Prediction / Analytics	PASS
FT-12	Functional	GET /api/kpi/submissions returns all KPI submissions	Submission / Retrieval	PASS

FT-1 3	Functional	POST /api/kpi/verify-submission approves staff submission	Submission / Approval	PASS
FT-1 4	Functional	POST /api/kpi/verify-submission rejects staff submission	Submission / Rejection	PASS
FT-1 5	Functional	GET /api/kpi/evidence/{file_name} retrieves evidence file and handles missing file	File Access / Evidence	PASS

Test ID	Type	Description	Category	Status
UT-01	Unit	Transaction array structure (length, properties)	Data Validation	PASS
UT-02	Unit	Warning condition logic — amount > 100	Logic/Boundary	PASS
UT-03	Unit	EJS view engine configuration	Configuration	PASS
UT-04	Unit	Transaction field data types	Data Validation	PASS
FT-01	Functional	GET / returns 200 and home page HTML	Route/HTTP	PASS
FT-02	Functional	GET /transaction/:month renders all 3 rows	Route/Rendering	PASS
FT-03	Functional	Warning message rendered for amount > 100	Rendering/Logic	PASS
FT-04	Functional	Unknown routes return 404	Error Handling	PASS
FT-05	Functional	Static CSS file served from /app.css	Static Files	PASS
FT-06	Functional	Route param :month passed to EJS view	Route/Data	PASS

4.2 Test Coverage Summary

Category	Total Tests	Passed	Failed
Unit Tests	5	5	0
Functional Tests	6	6	0
TOTAL	11	11	0

4.3 index.js Export Setup for Testing

To make the Express app and transactions array accessible to Jest, export them from index.js:

```
// Add at the bottom of index.js
module.exports = { app, transactions };
```

Then in your test files, import using:

```
const { app, transactions } = require('./index');
const request = require('supertest');
```

4.4 Running All Tests

Execute the full test suite with:

```
npm test
```

Expected output:

```
PASS tests/unit/test_profile.py
UT-01 ProfileUpdate Model Validation
  ✓ All fields provided
  ✓ Partial fields provided
  ✓ No fields provided

UT-02 PasswordUpdate Model Validation
  ✓ Valid password object
  ✓ Required fields exist

UT-03 User Profile Document Builder
  ✓ Standard profile document
  ✓ Missing phone defaults to empty string
  ✓ Public document adds id

UT-04 Password Hashing and Verification
  ✓ Hash creation
  ✓ Correct password check
  ✓ Wrong password check

UT-05 Update Password Service Logic
  ✓ Password mismatch rejected
  ✓ Valid password change accepted
  ✓ Wrong current password rejected

PASS tests/functional/test_profile_routes.py
FT-01 GET /api/user
  ✓ Returns 200 for valid token
  ✓ Profile fields returned correctly
```

✓ User not found returns 404

FT-02 GET /api/users

- ✓ Returns 200
- ✓ Returns user array
- ✓ Each item has required fields

FT-03 PUT /api/profile

- ✓ Updates profile successfully
- ✓ Partial update works
- ✓ User not found returns 404

FT-04 PUT /api/password

- ✓ Password change succeeds
- ✓ Password mismatch returns 400
- ✓ Wrong current password returns 401

FT-05 DELETE /api/profile

- ✓ Deletes account successfully
- ✓ Missing user returns 404
- ✓ Auth record cleanup verified

FT-06 GET /api/staff

- ✓ Only staff users returned
- ✓ Required fields present
- ✓ Empty staff list handled

Test Suites: 2 passed, 2 total

Tests: 11 passed, 11 total

PASS __tests__/unit.test.js

Transaction Array Structure

- ✓ Array contains 3 transactions
- ✓ Each transaction has date, type, amount
- ✓ amount is a number in all records

Warning Condition Logic

- ✓ Amount 30 does NOT trigger warning
- ✓ Amount 123 TRIGGERS warning

PASS __tests__/functional.test.js

FT-01: GET /

- ✓ Returns 200 status code
- ✓ Content-Type is text/html

...

Test Suites: 2 passed, 2 total

Tests: 10 passed, 10 total

python -m pytest tests -v

Actual output:

```
PS C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend> python -m pytest tests/functional/test_manager_routes.py -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\peiju\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend
plugins: anyio-4.13.0
collected 15 items

tests/functional/test_manager_routes.py::test_ft01_manager_endpoint_missing_jwt_token PASSED [ 6%]
tests/functional/test_manager_routes.py::test_ft02_manager_endpoint_staff_token_forbidden PASSED [ 13%]
tests/functional/test_manager_routes.py::test_ft03_create_kpi_success PASSED [ 20%]
tests/functional/test_manager_routes.py::test_ft04_create_kpi_validation_error PASSED [ 26%]
tests/functional/test_manager_routes.py::test_ft05_update_kpi_success PASSED [ 33%]
tests/functional/test_manager_routes.py::test_ft06_delete_kpi_success PASSED [ 40%]
tests/functional/test_manager_routes.py::test_ft07_retrieve_all_kpis PASSED [ 46%]
tests/functional/test_manager_routes.py::test_ft08_retrieve_single_kpi_success_and_not_found PASSED [ 53%]
tests/functional/test_manager_routes.py::test_ft09_retrieve_dashboard_stats PASSED [ 60%]
tests/functional/test_manager_routes.py::test_ft10_retrieve_kpi_history_data PASSED [ 66%]
tests/functional/test_manager_routes.py::test_ft11_retrieve_kpi_prediction PASSED [ 73%]
tests/functional/test_manager_routes.py::test_ft12_retrieve_all_submissions PASSED [ 80%]
tests/functional/test_manager_routes.py::test_ft13_approve_staff_submission PASSED [ 86%]
tests/functional/test_manager_routes.py::test_ft14_reject_staff_submission PASSED [ 93%]
tests/functional/test_manager_routes.py::test_ft15_download_evidence_file_success_and_not_found PASSED [100%]

===== warnings summary =====
tests/functional/test_manager_routes.py::test_ft03_create_kpi_success
  C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend\services\kpi_service.py:107: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.utcnow().isoformat()

tests/functional/test_manager_routes.py::test_ft03_create_kpi_success
  C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend\services\kpi_service.py:119: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    "kpiAssignments": [a.dict() for a in kpi_data.kpiAssignments] if kpi_data.kpiAssignments else [],

tests/functional/test_manager_routes.py::test_ft05_update_kpi_success
  C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend\services\kpi_service.py:183: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    update_fields['updatedAt'] = datetime.utcnow().isoformat()

tests/functional/test_manager_routes.py::test_ft10_retrieve_kpi_history_data
  C:\Users\peiju\OneDrive\Documents\kpi_tracker\kpi_tracker\backend\services\kpi_service.py:527: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    today = datetime.utcnow()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 15 passed, 4 warnings in 44.54s =====
```